

The Lab Manual: OpenShift Virtualization and VMware Migration

Field manual, migration edition. Linux, networking, OpenShift 4.22, OpenShift Virtualization and MTV, learned by running real commands on your own cluster.

Gaganpreet Singh · 5 October 2026

RHEL 10.0 bastion

OpenShift 4.22.15

Kubernetes 1.35.6

RHCOS 9.8

OpenShift Virtualization 4.22.9

MTV 2.12.9

NMState 4.22

csi-driver-nfs 4.11

vSphere 8

CONTENTS

How to use this book

- The capture lab
- Conventions
- The lab at a glance
- Rebuilding the lab from zero

Chapter 1 · Linux for the Migration Engineer (RHEL 10)

- Know the box you are on
- Disks, partitions and filesystems
- Packages without a subscription
- Services, firewall and the NFS server
- Text tools you will use against `oc` output
- Check yourself

Chapter 2 · Networking for the Cluster and the Migration

- Five ideas, in vSphere terms
- The bastion's view of the network
- DNS: the records OpenShift needs
- Ports: can I reach what I need?
- Time
- Check yourself

Chapter 3 · Installing OpenShift with the Agent-Based Installer

- Sizing the lab
- The two input files
- Build the ISO and boot
- First look at the cluster
- Check yourself

Chapter 4 · Day 1: Logging In, Users and Access

- Who am I, and where?
- Add a real login: HTPasswd
- Remove kubeadmin
- RBAC in one table
- Check yourself

Chapter 5 · Storage: NFS, CSI and Persistent Volumes

The NFS server (bastion side)
The CSI driver and StorageClass (cluster side)
Following a disk from PVC to folder
Check yourself

Chapter 6 · OpenShift Virtualization: VMs as Pods

Install: three operators
Is hardware virtualization really there?
Boot sources, instance types and preferences
Creating, starting and stopping a VM
The API you are now using
Check yourself

Chapter 7 · VM Networking: NMState, Bridges and VLANs

Layer 2: the bridge (NMState)
Layer 3: the VM networks (NADs)
Check yourself

Chapter 8 · Migration Toolkit for Virtualization (MTV)

The objects, in the order you create them
The operator
Step 1 · A vCenter account for MTV
Step 2 · The source provider
Step 3 · Network and storage maps
Step 4 · What a Plan can do
Check yourself

Chapter 9 · End to End: Migrating `ubuntu-noble-24.04-cloudimg`

9.1 Create the target project
9.2 The plan, in the web console
9.3 The same plan with `oc`
9.4 Start it: the Migration object
9.5 Watching the pipeline
9.6 What MTV built
9.7 First boot
9.8 Validate, roll back, clean up
Check yourself

Chapter 10 · Troubleshooting and Going to Production

The troubleshooting order

Symptom table

Support data

From lab to production

Check yourself

Appendix A · Command Cheat Sheets

Cluster

Storage

Virtualization

VM networking

MTV

Bastion (Linux)

Appendix B · Exam Map

How to use this book

NOTE

Placeholders. Every hostname, IP address, VLAN, datastore and port group in this public edition is a placeholder (`example.com` , `198.51.100.0/24` , `203.0.113.0/24`). The command output is real output from the author's lab build with those values substituted. Replace them with your own environment's values before running anything.

This is a hands-on field manual, written in the same style as *The Lab Manual (Complete Edition)*, but about one thing: moving virtual machines from VMware vSphere to Red Hat OpenShift Virtualization. It starts from a Linux box and an empty vSphere folder, and ends with a VMware VM booted as an OpenShift VM.

Every dark terminal window marked **real capture** was executed on the author's lab on 5 October 2026 and the output is pasted exactly as the system printed it (only very long outputs are trimmed, marked `...`). Every command in a real capture is read-only, so you can re-run any of them at any time without changing anything.

Blocks marked **Reference** are the commands that *changed* the lab: installing the cluster, creating the NFS export, installing the operators, creating the bridge, adding the vCenter provider, starting the migration. They are the exact scripts that were run (they live in `openshift-manual/install/` and `openshift-manual/migration/`), shown as reference because re-running them on a working cluster is either pointless or harmful.

Screenshots are marked with a dashed **Screenshot** box that names the exact screen and what to look for. Save your capture as `images/S-nn.png` next to the book and run `python3 build.py` : the picture replaces the box in the HTML, Word and PDF editions.

The capture lab

LAYER	WHAT RAN	WHY IT MATTERS TO YOU
Source	VMware vSphere 8, vCenter <code>vcenter</code> , ESXi <code>esxi01 .. esxi04</code>	The platform you are migrating from
Bastion	RHEL 10.0 VM <code>bastion.example.com</code> (198.51.100.24), 8 vCPU, 16 GiB, 1 TiB + 500 GiB	Your Linux workstation: <code>oc</code> , installer, NFS server
Cluster	OpenShift 4.22.15 compact cluster <code>ocp1</code> , 3 nodes (12 vCPU, 48 GiB each), agent-based install	Every node is control plane <i>and</i> worker
Storage	NFS export <code>/exports/ocp</code> on the bastion + <code>csi-driver-nfs</code> v4.11.0, StorageClass <code>nfs-csi</code> (RWX)	RWX disks make VM live migration possible
Virtualization	OpenShift Virtualization 4.22.9 (KubeVirt), nested virtualization on vSphere	Runs VMs as Pods
VM networking	Kubernetes NMState 4.22, linux bridge <code>br-vm</code> on NIC2, VLAN NADs <code>vlan-110</code> , <code>vlan-120</code> , <code>vlan-130</code>	The OpenShift version of a port group
Migration	Migration Toolkit for Virtualization (MTV) 2.12.9, provider <code>vcenter-lab</code>	Copies and converts VMware VMs
Test VM	<code>ubuntu-noble-24.04-cloudimg</code> (2 vCPU, 1 GiB, 10 GiB disk)	Migrated cold, end to end, in Chapter 9

NOTE

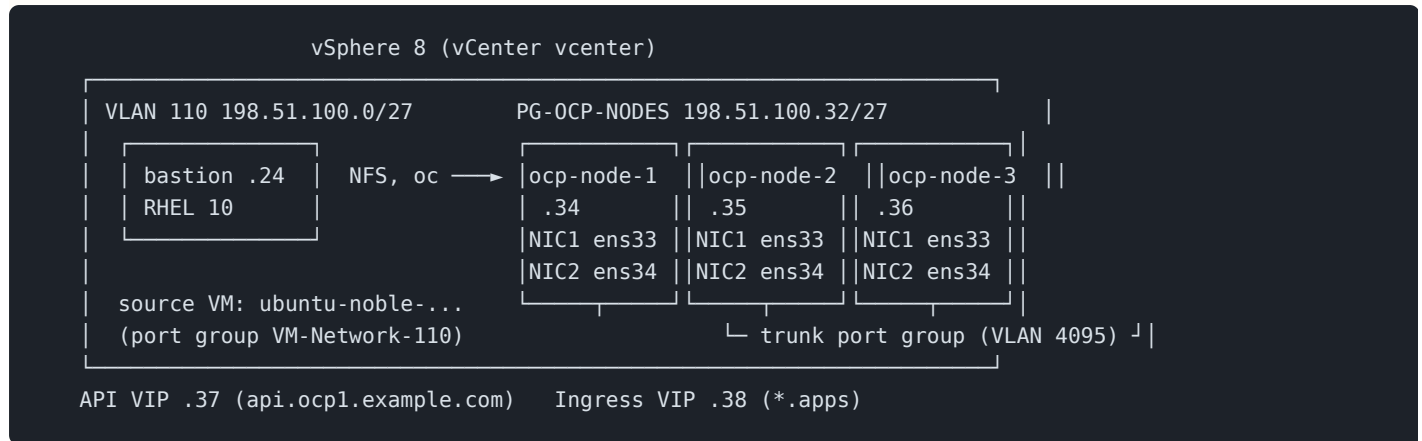
The cluster is a *proof of concept* on nested virtualization: OpenShift nodes are themselves VMware VMs. It behaves exactly like a production cluster for every command in this book, but it is slower (nested KVM) and its storage is one NFS export. Wherever production differs, a **PRODUCTION** box says how.

Conventions

- `[admin@bastion ~]$` = run on the bastion as your normal user. Commands that need root are prefixed `sudo`.
- `PS C:\>` = run in PowerShell with VMware PowerCLI on the Windows jump host.
- **NOTE** boxes explain, **TIP** boxes save time, **EXAM TIP** boxes map content to the Red Hat exams (EX316 OpenShift Virtualization, EX280 OpenShift Administration, EX188/DO188, DO180).
- **GOTCHA — REAL ISSUE** boxes are problems actually hit while building this lab. Each one is a troubleshooting lesson.
- **LAB — DO IT ON THE BASTION** boxes are exercises. **Check yourself** sections end each chapter: questions, answers, and the logic behind each answer.

- `<ANGLE_BRACKETS>` = a value you replace with your own.

The lab at a glance



Rebuilding the lab from zero

```
# Reference: the order this lab was built in. Each step is a chapter.
# 1. Bastion: RHEL 10 VM, second 500 GB disk, NFS export (Chapter 1, 5)
# 2. Network: three /27 VLANs, DNS records in AD, routes (Chapter 2)
# 3. vSphere: 3 node VMs, agent ISO, boot (Chapter 3)
~/ocp-install/build-agent-iso.sh
openshift-install agent wait-for install-complete --dir ~/ocp1
# 4. Day 1: htpasswd admin, remove kubeadmin (Chapter 4)
~/ocp-install/add-htpasswd-admin.sh
# 5. Storage and operators (Chapter 5, 6)
~/ocp-install/install-nfs-csi.sh
~/ocp-install/install-cnv-nmstate-mtv.sh
# 6. VM networks (Chapter 7)
~/ocp-install/vm-networks.sh
# 7. MTV provider, maps, plan, migration (Chapter 8, 9)
~/ocp-install/add-mtv-vsphere-provider.sh
oc apply -f ~/ocp-install/mtv-test/test-ubuntu-plan.yaml
oc apply -f ~/ocp-install/mtv-test/test-ubuntu-migration.yaml
```

Linux for the Migration Engineer (RHEL 10)

Every OpenShift node is a Linux box, every migrated VM disk lands in a Linux filesystem, and every `oc` command you type runs in a Linux shell. You do not need to become a Linux administrator to migrate VMs, but you need to be comfortable on the **bastion**: the RHEL 10 VM that holds the installer, the `oc` client, the cluster credentials and the NFS share.

Think of the bastion as the Windows jump host with PowerCLI you already use for vSphere. It is where you stand when you talk to the cluster.

Know the box you are on

Before touching anything, identify the distribution, the kernel and who you are.

```
admin@bastion - real capture

[admin@bastion ~]$ cat /etc/os-release | head -4; uname -r
NAME="Red Hat Enterprise Linux"
VERSION="10.0 (Coughlan)"
ID="rhel"
ID_LIKE="centos fedora"
6.12.0-55.9.1.el10_0.x86_64
```

```
admin@bastion - real capture

[admin@bastion ~]$ whoami; id; hostname; nproc; free -h | head -2
admin
uid=1000(admin) gid=1000(admin) groups=1000(admin),10(wheel) context=unconfined_u:unconfined_r:unconfine
d_t:s0-s0:c0.c1023
bastion.example.com
8

              total        used        free      shared  buff/cache   available
Mem:           15Gi         6.1Gi        224Mi       348Mi       9.7Gi        9.3Gi
```

How to read it:

- `ID="rhel"` with `VERSION="10.0"` tells you the package manager is `dnf`, the firewall is `firewalld`, and SELinux is on.
- `groups=...10(wheel)` means you may use `sudo`. Membership of `wheel` is the Linux version of being in the vCenter *Administrators* group.
- `context=unconfined_u:...` is your **SELinux** label. You will meet SELinux again on the nodes: it is why a container cannot read a file just because the Unix permissions allow it.
- `free` shows only 224 MiB "free" but 9.3 GiB "available". Linux uses spare RAM as file cache (`buff/cache`) and gives it back on demand. **Available** is the number that matters, the same

way you read *consumed* vs *active* memory on an ESXi host.

Disks, partitions and filesystems

```
admin@bastion - real capture

[admin@bastion ~]$ lsblk -o NAME,SIZE,TYPE,FSTYPE,LABEL,MOUNTPOINTS
NAME                SIZE TYPE FSTYPE      LABEL                MOUNTPOINTS
sda                  1T disk
├─sda1                600M part vfat                /boot/efi
├─sda2                 1G part xfs                /boot
└─sda3             1022.4G part LVM2_member
   ├─rhel-root         70G lvm  xfs                /
   ├─rhel-swap         7.9G lvm  swap              [SWAP]
   └─rhel-home       944.5G lvm  xfs                /home
sdb                  500G disk xfs              ocpnfs              /exports/ocp
sr0                   7.9G rom  iso9660          RHEL-10-0-BaseOS-x86_64 /mnt/rhel10dvd
```

LINUX	VSPHERE EQUIVALENT	IN THIS LAB
sda , sdb (disk)	A virtual disk (VMDK) attached to the VM	sda = 1 TB OS disk, sdb = 500 GB NFS disk
sda1 (partition)	A slice of that disk	EFI and /boot
LVM volume group rhel	A datastore spanning one or more disks	Holds root , swap , home
LVM logical volume rhel-root	A VMDK carved from the datastore	70 GB root filesystem
xfs (filesystem)	VMFS, but inside the guest	All data filesystems
Mount point /exports/ocp	A drive letter, except it is a folder	Where the NFS disk appears
sr0	The VM's CD/DVD drive	RHEL 10 DVD, used as the local package repository

```
admin@bastion - real capture

[admin@bastion ~]$ df -h /exports/ocp
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdb         500G   30G  470G   6% /exports/ocp
```

The 30 GB already used is the boot-source images OpenShift Virtualization downloaded (Chapter 6) plus the migrated Ubuntu disk (Chapter 9).

```
# Reference: how the 500 GB disk became /exports/ocp (run once, after adding the disk in vCenter)
sudo mkfs.xfs -L ocpnfs /dev/sdb
sudo mkdir -p /exports/ocp
echo 'LABEL=ocpnfs /exports/ocp xfs defaults 0 0' | sudo tee -a /etc/fstab
sudo systemctl daemon-reload && sudo mount -a
```

TIP

Mount by `LABEL=` (or `UUID=`), never by `/dev/sdb`. Device names can change order when you add a disk in vCenter; the label travels with the filesystem.

Packages without a subscription

The bastion is not registered with Red Hat, so `dnf` installs from the RHEL 10 DVD mounted at `/mnt/rhel10dvd` (the `sr0` line above).

```
# Reference: local repository from the DVD
sudo mkdir -p /mnt/rhel10dvd && sudo mount /dev/sr0 /mnt/rhel10dvd
sudo tee /etc/yum.repos.d/rhel10-dvd.repo <<'EOF'
[dvd-BaseOS]
name=RHEL 10 DVD BaseOS
baseurl=file:///mnt/rhel10dvd/BaseOS
gpgcheck=0
[dvd-AppStream]
name=RHEL 10 DVD AppStream
baseurl=file:///mnt/rhel10dvd/AppStream
gpgcheck=0
EOF
sudo dnf -y install nfs-utils nmstate httpd-tools bind-utils jq
```

Services, firewall and the NFS server

```
admin@bastion - real capture
```

```
[admin@bastion ~]$ sudo firewall-cmd --list-services
cockpit dhcpv6-client mountd nfs rpc-bind ssh
```

`nfs`, `mountd` and `rpc-bind` were opened for the NFS server; `cockpit` is the bastion's own web console on port 9090. A service in Linux (`systemctl status nfs-server`) is the equivalent of a Windows service; `firewalld` is the Windows Defender Firewall.

NEED	COMMAND
Is a service running?	<code>systemctl status nfs-server</code>
Start now and at every boot	<code>sudo systemctl enable --now nfs-server</code>
Last 50 log lines of a service	<code>journalctl -u nfs-server -n 50</code>
Open a firewall service permanently	<code>sudo firewall-cmd --permanent --add-service=nfs && sudo firewall-cmd --reload</code>
Which ports are listening	<code>sudo ss -tlnp</code>
Who uses the disk	<code>sudo du -sh /exports/ocp/* sort -h tail</code>
Find big files	<code>find / -xdev -size +1G -type f 2>/dev/null</code>
Follow a log live	<code>journalctl -f</code>

Text tools you will use against `oc` output

`oc` prints tables. Five Linux tools turn those tables into answers.

```
oc get pods -A --no-headers | awk '{print $4}' | sort | uniq -c      # how many Pods in each state
oc get pods -A --no-headers | grep -v -E 'Running|Completed'      # only the unhealthy ones
oc get co --no-headers | awk '$3!="True" || $5!="False"'           # degraded cluster operators
oc get pv --no-headers | awk '{s+=$2} END {print s/1024/1024/1024 " GiB"}' # total PV size
oc get vm -A -o name | xargs -n1 basename                          # just the VM names
```

GOTCHA — REAL ISSUE

PowerShell on the Windows jump host rewrites double quotes inside the argument of `ssh host "command"`. A `jsonpath` like `{"\n"}` arrives on the bastion broken. Two fixes were used while building this lab: put the commands in a script on the bastion and run the script, or send it base64-encoded (`echo <base64> | base64 -d | bash`). From inside an SSH session on the bastion the problem does not exist.

EXAM TIP

Red Hat exams run on RHEL. You will be expected to use `vi / vim`, `grep`, `systemctl`, `journalctl`, `firewall-cmd`, `ss`, `lsblk`, `df` and `dnf` without help. DO180 and EX280 assume this Linux floor; EX316 assumes EX280.

LAB — DO IT ON THE BASTION

(1) Find how much space the migrated Ubuntu disk takes under `/exports/ocp` with `sudo du -sh /exports/ocp/*`. (2) Show which process listens on TCP 2049 with `sudo ss -tlnp | grep 2049`. (3) Count Pods per state across the cluster with the `awk | sort | uniq -c` pattern above.

Check yourself

Q1. `free -h` shows 224 MiB free. Is the bastion out of memory? No. *Logic:* Linux keeps spare RAM as page cache (`buff/cache` 9.7 GiB) and frees it on demand. The real headroom is the **available** column, 9.3 GiB.

Q2. You add a third disk in vCenter and after a reboot `/exports/ocp` is empty. Why, and how do you prevent it? The new disk may have taken the name `sdb` and the NFS disk became `sdc`, so a mount by device name mounted the wrong (empty) disk. *Logic:* device names follow detection order; labels and UUIDs belong to the filesystem. Always mount by `LABEL=` or `UUID=`.

Q3. Which group membership lets `admin` use `sudo` here, and what is the vSphere analogy? `wheel` (gid 10). *Logic:* RHEL's sudoers grants `wheel` full sudo, the same way a vCenter role granted on the root object applies everywhere.

Q4. A client cannot mount the NFS share, but `systemctl status nfs-server` is active. What is the first thing to check on the bastion? The firewall: `sudo firewall-cmd --list-services` must show `nfs` (and `mountd`, `rpc-bind` for NFSv3 clients). *Logic:* a running service is useless if its port is closed; it is the Linux version of "the VM is up but the port group is wrong".

Networking for the Cluster and the Migration

Most failed OpenShift installs and most failed migrations are network problems: a missing DNS record, a route that does not exist, a firewall in the path, or a VLAN that is not on the trunk. This chapter gives you the five networking ideas you need and shows each one on the real lab.

Five ideas, in vSphere terms

IDEA	WHAT IT IS	VSPHERE YOU ALREADY KNOW
IP address + prefix	<code>198.51.100.34/27</code> = address <code>.34</code> in a network of 32 addresses	The IP you type in the VM's guest customization
Gateway	The router address that leads out of your subnet	The default gateway in the same screen
VLAN	A numbered virtual network on the same cable	The VLAN ID on a port group
Route	"To reach network X, go through router Y"	ESXi has these too (<code>esxcli network ip route</code>)
DNS	Name to IP lookup	How vCenter finds <code>esx001.example.com</code>

Reading a /27

A prefix says how many bits are the network. `/24` = 256 addresses, `/27` = 32 addresses. The network team gave this lab three /27 VLANs:

NAME	VLAN	NETWORK	GATEWAY	USED FOR
PG-OCP-NODES	100	198.51.100.32/27 (.32 to .63)	.33	OpenShift nodes .34 to .36, API VIP .37, Ingress VIP .38
PG-VM-120	120	198.51.100.64/27 (.64 to .95)	.65	VM network NAD <code>vlan-120</code>
PG-VM-130	130	198.51.100.96/27 (.96 to .127)	.97	VM network NAD <code>vlan-130</code>
110 (old)	110	198.51.100.0/27	.1	Bastion and the source VM's port group

The bastion's view of the network

```

admin@bastion - real capture

[admin@bastion ~]$ ip -br addr
lo                UNKNOWN      127.0.0.1/8  ::1/128
ens33             UP          198.51.100.24/24  fe80::250:56ff:fe00:1/64

```

```

admin@bastion - real capture

[admin@bastion ~]$ ip route
default via 198.51.100.1 dev ens33 proto static metric 100
198.51.100.0/24 dev ens33 proto kernel scope link src 198.51.100.24 metric 100
198.51.100.32/27 via 198.51.100.1 dev ens33 proto static metric 100
198.51.100.64/27 via 198.51.100.1 dev ens33 proto static metric 100
198.51.100.96/27 via 198.51.100.1 dev ens33 proto static metric 100

```

How to read it, line by line:

1. `default via 198.51.100.1` : anything not listed below goes to the router `.1`.
2. `198.51.100.0/24 dev ens33 ... scope link` : the bastion believes **all** of 198.51.100.x is directly on its own wire. That was true before the network team cut the /24 into /27 VLANs. It is not true any more.
3. The three `/27 via 198.51.100.1` routes are the fix. Linux always picks the **most specific** match, so traffic to `.34` matches `/27` (27 bits) before `/24` (24 bits) and goes through the router, which knows where VLAN 100 is.

GOTCHA — REAL ISSUE

After the nodes moved to PG-OCP-NODES the bastion could not reach them: it tried to ARP for `.34` on its own wire (because of the old `/24` mask) and nobody answered. Adding the three static routes fixed it without touching the bastion's own IP: `sudo nmcli connection modify ens33 +ipv4.routes "198.51.100.32/27 198.51.100.1, 198.51.100.64/27 198.51.100.1, 198.51.100.96/27 198.51.100.1" && sudo nmcli connection up ens33`

```
admin@bastion - real capture

[admin@bastion ~]$ ping -c 2 -W 2 198.51.100.33 | tail -2
2 packets transmitted, 2 received, 0% packet loss, time 1017ms
rtt min/avg/max/mdev = 0.525/0.544/0.563/0.019 ms
```

The node-network gateway answers in about half a millisecond: the route works.

DNS: the records OpenShift needs

An OpenShift cluster needs exactly these names before installation. In this lab they are A records in the Active Directory DNS zone `example.com`, under the cluster name `ocp1`.

```
admin@bastion - real capture

[admin@bastion ~]$ grep nameserver /etc/resolv.conf
nameserver 203.0.113.251
nameserver 203.0.113.252
```

```
admin@bastion - real capture

[admin@bastion ~]$ for h in api api-int test.apps ocp-node-1 ocp-node-2 ocp-node-3; do printf "%-12s %s\n" $h $(dig +short $h.ocp1.example.com); done
api           198.51.100.37
api-int       198.51.100.37
test.apps     198.51.100.38
ocp-node-1    198.51.100.34
ocp-node-2    198.51.100.35
ocp-node-3    198.51.100.36
```

RECORD	POINTS TO	WHO USES IT
api.<cluster>.<domain>	API VIP .37	You, oc , the web console
api-int.<cluster>.<domain>	API VIP .37	The nodes themselves
*.apps.<cluster>.<domain> (wildcard)	Ingress VIP .38	Every Route: console, OAuth, MTV UI, your apps
node names	node IPs	Nice to have; the installer uses the MACs and IPs

test.apps resolving to .38 proves the **wildcard** works: nobody created a record called test .

```
# Reference: how the records were created (PowerShell on the jump host, AD DNS)
PS C:\> $z='example.com'; $s='dc01.example.com'
PS C:\> Add-DnsServerResourceRecordA -ComputerName $s -ZoneName $z -Name 'api.ocp1' -IPv4Address 19
8.51.100.37
PS C:\> Add-DnsServerResourceRecordA -ComputerName $s -ZoneName $z -Name 'api-int.ocp1' -IPv4Address 19
8.51.100.37
PS C:\> Add-DnsServerResourceRecordA -ComputerName $s -ZoneName $z -Name '*.apps.ocp1' -IPv4Address 19
8.51.100.38
PS C:\> 1..3 | % { Add-DnsServerResourceRecordA -ComputerName $s -ZoneName $z -Name "ocp-node-$_ocp1" -
IPv4Address "198.51.100.$(33+$_)" }
```

GOTCHA — REAL ISSUE

The first plan named the cluster ocp , giving api.ocp.example.com . That name already belonged to another team's cluster in the same AD zone. Creating the record would have pointed their API at your VIP. Always nslookup api.<name>.<domain> **before** choosing a cluster name; this lab became ocp1 for that reason.

GOTCHA — REAL ISSUE

The bastion originally used public DNS (4.2.2.2, 8.8.8.8). Public DNS cannot resolve example.com , so openshift-install wait-for and oc could not find api.ocp1.example.com . Setting the bastion's DNS to the domain controllers (nmcli connection modify ens33 ipv4.dns "203.0.113.251 203.0.113.252") fixed it.

```
admin@bastion - real capture

[admin@bastion ~]$ dig +short -x 198.51.100.37; dig +short vcenter.example.com
203.0.113.9
```

The first query (reverse lookup of the API VIP) returned nothing: there is **no PTR record**. OpenShift does not need one. The second shows vCenter at 203.0.113.9 , which MTV will need to resolve from inside the cluster.

Ports: can I reach what I need?

MTV talks to vCenter on 443 (inventory, and disk reads without VDDK) and to each ESXi host on 902 (disk reads with VDDK). Test before you migrate, not during.

```
admin@bastion - real capture

[admin@bastion ~]$ for t in vcenter.example.com:443 esxi01.example.com:902 esxi02.example.com:902 api.oc
pl.example.com:6443; do timeout 4 bash -c "</dev/tcp/${t%:*}/${t#*:}" && echo "$t open" || echo "$t CLOS
ED"; done
vcenter.example.com:443 open
esxi01.example.com:902 open
esxi02.example.com:902 open
api.ocpl.example.com:6443 open
```

TIP

`bash -c "</dev/tcp/HOST/PORT"` is a port test that needs no extra package; it is the Linux `Test-NetConnection -Port . nc -zv HOST PORT` does the same if `nmap-ncat` is installed.

FROM	TO	PORT	WHY
Bastion	API VIP	6443	oc
Your browser	Ingress VIP	443	Web console, MTV UI
OpenShift nodes	vCenter	443	MTV inventory and (without VDDK) disk data
OpenShift nodes	every ESXi host	902	VDDK disk data (NBD)
OpenShift nodes	NFS server	2049	Persistent volumes
OpenShift nodes	NTP	UDP 123	Time; etcd and certificates break without it

Time

```
admin@bastion - real capture

[admin@bastion ~]$ chronyc sources 2>/dev/null | head -8
MS Name/IP address         Stratum Poll Reach LastRx Last sample
=====
^? 99-28-14-242.lightspeed.> 0 10    0    -    +0ns[  +0ns] +/-    0ns
^? time.cloudflare.com      0 10    0    -    +0ns[  +0ns] +/-    0ns
^? h134-215-155-177.mdnwi.> 0 10    0    -    +0ns[  +0ns] +/-    0ns
^? time2.trtnw.net          0 10    0    -    +0ns[  +0ns] +/-    0ns
^? tick.chil.ntfo.org       0 10    0    -    +0ns[  +0ns] +/-    0ns
^? owners.kjssl.com         0 10    0    -    +0ns[  +0ns] +/-    0ns
```

GOTCHA — REAL ISSUE

This capture is a live problem, not a historic one. `^?` and `Reach 0` mean the bastion has **never** reached any of its public NTP servers: UDP 123 to the internet is blocked from 198.51.100.x. The cluster nodes do not have this problem because their agent config lists the lab NTP servers (203.0.113.117 , .118 and the two DCs, Chapter 3). Fix the bastion the same way: replace the `pool` line in `/etc/chrony.conf` with `server 203.0.113.117 iburst` and `server 203.0.113.118 iburst` , then `sudo systemctl restart chronyd` and check that `^*` appears.

GOTCHA — REAL ISSUE

VMs on ESXi host `esxi04` lose their network in this lab. A DRS *must not run on* rule keeps the bastion, the OCP nodes and the source VMs off it (a DRS rule script). If a migration or a node suddenly cannot reach anything, check which host it runs on before debugging Linux.

EXAM TIP

EX280 and EX316 do not ask you to build DNS, but they do expect you to diagnose it: `oc get route` , `dig` , `curl -kv https://<route-host>` , and reading why a Route has no endpoints.

LAB — DO IT ON THE BASTION

(1) Prove that `random-name.apps.ocp1.example.com` resolves to `.38` . (2) Remove nothing, but explain with `ip route get 198.51.100.34` which route the kernel picks and why. (3) Fix the bastion's NTP as described above and capture `chronyc sources` again.

Check yourself

Q1. The bastion has 198.51.100.0/24 dev ens33 and also 198.51.100.32/27 via 198.51.100.1 . Which one is used for 198.51.100.35? The `/27` route. *Logic:* longest-prefix match. 27 matching bits beat 24, so the packet goes to the router instead of being ARPed on the local wire.

Q2. Why does OpenShift need a wildcard record and not one record per application?

Every Route gets a new host name under `*.apps` , created at any time by any user. *Logic:* the Ingress VIP is the single entry point; the router (HAProxy) reads the host name in the request and forwards it, so DNS only needs to send all names to that one VIP.

Q3. api and api-int point to the same VIP. Why have two names? `api` is for clients, `api-int` for the nodes. *Logic:* in larger designs they can point to different load balancers (external vs internal) without changing any node configuration.

Q4. Port 902 to `esxi03` is closed but 443 to vCenter is open. Will a migration work?

Without VDDK, yes (disk data comes through vCenter on 443, slowly). With VDDK, VMs on `esxi03` will fail at disk transfer. *Logic:* VDDK opens NBD connections directly to the ESXi host that owns the VM, on 902.

Q5. `chronyc sources` shows `^?` with `Reach 0` for every source. What does that mean and why should you care before installing OpenShift?

No source has ever answered. *Logic:* OpenShift certificates and etcd leader election depend on agreed time; nodes that drift apart fail TLS checks and etcd health. Give the nodes reachable NTP servers at install time.

Installing OpenShift with the Agent-Based Installer

OpenShift Container Platform (OCP) is Kubernetes plus everything a platform needs: an image registry, Routes, OAuth login, a web console, monitoring, and **Operators** that install and upgrade every component. The nodes run **Red Hat Enterprise Linux CoreOS (RHCOS)**, an immutable OS that the cluster manages itself. You do not SSH in and edit nodes; you change them through the API.

There are several ways to install. This lab uses the **agent-based installer**: you describe the cluster in two YAML files, the installer turns them into one bootable ISO, and you boot every node VM from it. No DHCP, no PXE, no load balancer appliance. It is the closest thing OpenShift has to "deploy an OVA".

Sizing the lab

ROLE	COUNT	VCPU	RAM	DISKS	NICS
Control plane + worker ("compact")	3	12	48 GiB	120 GB OS (<code>sda</code>) + 100 GB spare (<code>sdb</code>)	NIC1 PG-OCP-NODES, NIC2 VM trunk
Bastion (existing)	1	8	16 GiB	1 TB + 500 GB NFS	port group VM-Network-110

NOTE

A **compact** cluster has three nodes that are all control plane *and* worker. It is the smallest highly-available OpenShift, and the usual choice for a proof of concept. OpenShift Virtualization needs **hardware virtualization** inside the nodes, so on vSphere the node VMs must have *Expose hardware assisted virtualization to the guest OS* ticked (nested virtualization). Chapter 6 proves it worked.

The two input files

```
# Reference: install-config.yaml (the cluster)
apiVersion: v1
baseDomain: example.com
metadata:
  name: ocp1          # becomes api.ocp1.example.com
compute:
  - name: worker
    replicas: 0          # compact: no separate workers
controlPlane:
  name: master
  replicas: 3
networking:
  networkType: OVNKubernetes
  machineNetwork:
    - cidr: 198.51.100.32/27      # the node VLAN PG-OCN-NODES
  clusterNetwork:
    - cidr: 10.128.0.0/14        # Pod IPs (internal only)
      hostPrefix: 23
  serviceNetwork:
    - 172.30.0.0/16             # Service IPs (internal only)
platform:
  baremetal:                  # "bring your own machines", VIPs handled by keepalived
    apiVIPs: [198.51.100.37]
    ingressVIPs: [198.51.100.38]
pullSecret: 'PULL_SECRET'    # filled in by build-agent-iso.sh, never committed
sshKey: 'SSH_KEY'
```

```
# Reference: agent-config.yaml (the machines), first host shown
apiVersion: v1beta1
kind: AgentConfig
metadata:
  name: ocp1
rendezvousIP: 198.51.100.34    # this node runs the installer service for the others
additionalNTPSources: [203.0.113.117, 203.0.113.118, 203.0.113.251, 203.0.113.252]
hosts:
  - hostname: ocp-node-1
    role: master
    rootDeviceHints: {deviceName: /dev/sda}
    interfaces:
      - {name: ens192, macAddress: 00:50:56:aa:00:11} # NIC1, matched by MAC
      - {name: ens224, macAddress: 00:50:56:aa:00:12} # NIC2, left without IP
    networkConfig:              # nmstate syntax: static IP, DNS, default route
      interfaces:
        - name: ens192
          type: ethernet
          state: up
          mac-address: 00:50:56:aa:00:11
          ipv4: {enabled: true, dhcp: false, address: [{ip: 198.51.100.34, prefix-length: 27}]}
          dns-resolver: {config: {server: [203.0.113.251, 203.0.113.252]}}
          routes:
            config: [{destination: 0.0.0.0/0, next-hop-address: 198.51.100.33, next-hop-interface: ens192}]
# ocp-node-2 (.35) and ocp-node-3 (.36) follow the same pattern with their own MACs
```

FIELD	MEANING	VSPHERE ANALOGY
machineNetwork	Where the node IPs live	The port group's subnet
clusterNetwork / serviceNetwork	Private overlay ranges inside the cluster	NSX segments that never leave the hosts
apiVIPs / ingressVIPs	Floating IPs moved between nodes by keepalived	A VIP on a load balancer appliance
rendezvousIP	The node that coordinates the install	The VCSA deployment stage 1
macAddress	How the ISO knows which config belongs to which VM	The MAC in the VM's NIC settings

GOTCHA — REAL ISSUE

The agent config names the NICs `ens192` / `ens224` , but on these VMs RHCOS calls them `ens33` / `ens34` (you will see that in Chapter 7). The install still worked because every interface is matched by **MAC address**, not by name. Lesson: always copy MACs from vCenter into the agent config; names are a hint, MACs are the key.

Build the ISO and boot

```
# Reference: on the bastion (needs ~/pull-secret.json from console.redhat.com, never stored in Git)
[admin@bastion ~]$ ~/ocp-install/build-agent-iso.sh
# -> ~/ocpl/agent.x86_64.iso
```

```
# Reference: on the jump host: upload the ISO, attach to the three VMs, power on
PS C:\> .\Mount-OcpAgentIso.ps1 -VCenter vcenter.example.com -Replace # copies the ISO from the bastion, uploads, attaches, powers on
```

```
# Reference: watch from the bastion
[admin@bastion ~]$ openshift-install agent wait-for bootstrap-complete --dir ~/ocpl
[admin@bastion ~]$ openshift-install agent wait-for install-complete --dir ~/ocpl
```

● ● ● admin@bastion – real capture

```
[admin@bastion ~]$ tail -4 ~/ocp-install/download.log
openshift-install 4.22.15
built from commit cda698d75d93e6ed9bf6203c988c5321a086758
release image quay.io/openshift-release-dev/ocp-release@sha256:fed788eac1c99388dd9b78dda4d6a73e39b70abb00ca918d2bb456a97187f0c1
release architecture amd64
```

The installer binary *is* the version: `openshift-install 4.22.15` can only install 4.22.15, from the release image digest shown.

SCREENSHOT S-01:

vSphere Client, folder *OCP-NODES*: the three VMs `OCP-NODE-1/2/3`, each with the agent ISO in CD/DVD drive 1 and *Expose hardware assisted virtualization* ticked under CPU.

SCREENSHOT S-02:

Console of `OCP-NODE-1` during install: the agent TUI showing the rendezvous host and the three hosts discovered.

GOTCHA — REAL ISSUE

After the install, the ISO was detached and NIC2 moved while the VMs were running. `ocp-node-1` was stunned for about eight minutes (its journal is silent from 12:46 to 12:54 UTC, with no reboot, then a burst of catch-up messages). A stunned control-plane node can cost etcd its quorum. Detach ISOs and change NICs with the node **powered off**, one node at a time.

First look at the cluster

admin@bastion — real capture

```
[admin@bastion ~]$ oc version
Client Version: 4.22.15
Kustomize Version: v5.7.1
Server Version: 4.22.15
Kubernetes Version: v1.35.6
```

admin@bastion — real capture

```
[admin@bastion ~]$ oc get nodes -o wide
NAME          STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE
KERNEL-VERSION   CONTAINER-RUNTIME
ocp-node-1    Ready     control-plane,master,worker  13h   v1.35.6   198.51.100.34  <none>        Red Hat Enterprise Linux CoreOS 9.8.20260915-0 (Plo
4.22.giteaf45bc.el9 5.14.0-687.48.1.el9_8.x86_64 cri-o://1.35.8-11.rhaos
ocp-node-2    Ready     control-plane,master,worker  13h   v1.35.6   198.51.100.35  <none>        Red Hat Enterprise Linux CoreOS 9.8.20260915-0 (Plo
4.22.giteaf45bc.el9 5.14.0-687.48.1.el9_8.x86_64 cri-o://1.35.8-11.rhaos
ocp-node-3    Ready     control-plane,master,worker  13h   v1.35.6   198.51.100.36  <none>        Red Hat Enterprise Linux CoreOS 9.8.20260915-0 (Plo
4.22.giteaf45bc.el9 5.14.0-687.48.1.el9_8.x86_64 cri-o://1.35.8-11.rhaos
```

Every node has all three roles (compact). The OS is RHCOS 9.8, the runtime is CRI-O (the container engine; there is no Docker on OpenShift).

```
admin@bastion - real capture

[admin@bastion ~]$ oc get clusterversion
NAME      VERSION  AVAILABLE  PROGRESSING  SINCE   STATUS
version   4.22.15  True       False        13h     Cluster version is 4.22.15
```

```
admin@bastion - real capture

[admin@bastion ~]$ oc get co
NAME                                     VERSION  AVAILABLE  PROGRESSING  DEGRADED  SINCE  MESSAGE
E
authentication                         4.22.15  True       False        False     8h
baremetal                             4.22.15  True       False        False     13h
cloud-controller-manager               4.22.15  True       False        False     13h
...
console                               4.22.15  True       False        False     13h
...
etcd                                   4.22.15  True       False        False     13h
image-registry                        4.22.15  True       False        False     13h
ingress                               4.22.15  True       False        False     13h
...
kube-apiserver                        4.22.15  True       False        False     13h
...
machine-config                        4.22.15  True       False        False     13h
...
network                               4.22.15  True       False        False     13h
...
openshift-apiserver                   4.22.15  True       False        False     8h
...
storage                               4.22.15  True       False        False     13h
```

All 34 **ClusterOperators** are `AVAILABLE=True` , `PROGRESSING=False` , `DEGRADED=False` : the cluster is healthy. A ClusterOperator is one platform component (DNS, ingress, etcd...) and its own operator reports its health here. This is the first command to run on any OpenShift cluster, the way you open *Monitor > Issues and Alarms* in vCenter. `authentication` and `openshift-apiserver` show `8h` instead of `13h` because they rolled out again when the htpasswd login was added (Chapter 4).

```
admin@bastion - real capture

[admin@bastion ~]$ oc get mcp
NAME    CONFIG                                UPDATED  UPDATING  DEGRADED  MACHINECOUNT
READYMACHINECOUNT  UPDATEDMACHINECOUNT  DEGRADEDMACHINECOUNT  AGE
master   rendered-master-037aa616469dfd56bcb6da3a1c53e738  True     False     False     3
3
worker   rendered-worker-86954aea22b2b3fd68fa55f7dfd3eef1  True     False     False     0
0
```

A **MachineConfigPool** groups nodes that get the same OS configuration. On a compact cluster all three nodes are in `master` and the `worker` pool is empty (`MACHINECOUNT 0`). That is normal, and it matters later: an OS change (a MachineConfig) aimed at the `worker` pool would do nothing here; aim it at `master` .

```

admin@bastion - real capture

[admin@bastion ~]$ oc get infrastructure cluster -o jsonpath="{.status.platform}{\"\n\"}{.status.apiServerURL}{\"\n\"}"; oc get network.config cluster -o jsonpath="{.status.networkType}{\"\n\"}{.status.clusterNetwork}{\"\n\"}{.status.serviceNetwork}{\"\n\"}"
BareMetal
https://api.ocp1.example.com:6443
OVNKubernetes
[{"cidr":"10.128.0.0/14","hostPrefix":23}]
["172.30.0.0/16"]

```

The cluster reads back exactly what `install-config.yaml` asked for. `BareMetal` is the platform type even though the nodes are VMware VMs: OpenShift does not talk to vCenter at all in this design, so it cannot create or delete node VMs. That is a choice. A `vsphere` platform install would let OpenShift create node VMs and VMDK-backed volumes itself.

```

admin@bastion - real capture

[admin@bastion ~]$ oc adm top nodes
NAME          CPU(cores)   CPU(%)   MEMORY(bytes)  MEMORY(%)
ocp-node-1    942m         8%       9213Mi         19%
ocp-node-2    1469m        12%      13384Mi        28%
ocp-node-3    1157m        10%      11042Mi        23%

```

`942m` = 942 **millicores**, a little under one CPU. An idle compact cluster with Virtualization and MTV installed uses about 1 core and 9 to 13 GiB per node: budget for that before sizing VMs.

PRODUCTION

Use separate infrastructure for control plane and VM workers (3 control plane + N bare-metal workers), size workers like ESXi hosts (the VMs' total vCPU and RAM plus 10 to 15 % for OpenShift), and install on bare metal, not nested. Nested virtualization is for labs only.

EXAM TIP

EX280 starts every task from a running cluster. Know `oc get clusterversion`, `oc get co`, `oc describe co <name>`, `oc get nodes`, `oc adm top nodes`, `oc get mcp` and `oc adm must-gather` by heart.

LAB — DO IT ON THE BASTION

(1) Run `oc describe co etcd` and find the line that lists the etcd members. (2) Run `oc get pods -n openshift-etcd` and count the etcd Pods: why three? (3) Run `oc get nodes --show-labels | tr , '\n' | grep node-role` and explain the three role labels.

Check yourself

Q1. The install-config says `compute replicas: 0` . Where do application Pods and VMs run? On the three control-plane nodes, which also carry the `worker` role. *Logic:* with zero workers the installer makes control-plane nodes schedulable (compact mode), shown in `ROLES control-plane, master, worker` .

Q2. You build a new ISO after changing an IP in `agent-config.yaml`, but the node still comes up with the old IP. What did you most likely forget? To boot from the **new** ISO (the old one was still in the datastore or still attached). *Logic:* the ISO carries the configuration; nothing reads the YAML after it is built.

Q3. One ClusterOperator shows `DEGRADED=True` . What do you run next? `oc describe co <name>` and read the `Conditions` messages, then look at the Pods in its namespace (`oc get pods -n openshift-<name>`). *Logic:* the operator reports the symptom; its Pods and events show the cause.

Q4. Why does `oc get mcp` show the `worker` pool with 0 machines, and when does that bite you? Compact cluster: all nodes are in `master` . *Logic:* MachineConfigs target pools by label; one labelled for `worker` will never roll out here, so you must label it `master` (or create a custom pool).

Q5. Why is the platform `BareMetal` when the nodes are vSphere VMs? The cluster was installed with the agent installer and `platform: baremetal` (no vCenter credentials). *Logic:* platform type decides what OpenShift integrates with; with `baremetal` it does not call vCenter, so storage and node VMs are managed outside the cluster.

Day 1: Logging In, Users and Access

The installer leaves you two ways in: a **kubeconfig** file with a certificate for `system:admin`, and a temporary user `kubeadmin` with a random password. Both are "break glass" credentials. Day 1 is replacing them with a real identity provider and a named admin.

Who am I, and where?

```

admin@bastion - real capture

[admin@bastion ~]$ oc whoami; oc whoami --show-server; oc whoami --show-console
system:admin
https://api.ocp1.example.com:6443
https://console-openshift-console.apps.ocp1.example.com

```

`system:admin` means `oc` is using the installer's certificate from `~/ocp1/auth/kubeconfig` (exported as `KUBECONFIG`). That file is the root password of the cluster: never copy it to a laptop, never commit it.

```

admin@bastion - real capture

[admin@bastion ~]$ oc config get-contexts
CURRENT  NAME                                     CLUSTER
AUTHINFO                                NAMESPACE
admin                                         ocp1
*      nfs-test/api-ocp1-example-com:6443/system:admin  api-ocp1-example-com:6443  system:admin/api-
-ocp1-example-com:6443  nfs-test

```

A **context** = cluster + user + current project. The `*` marks the active one, and its project is `nfs-test`.

GOTCHA — REAL ISSUE

`nfs-test` was a scratch project used to test NFS, and it was deleted afterwards. The context still pointed at it, and that broke a command you would never suspect. While capturing this book, `oc debug node/ocp-node-1` failed with error: unable to get namespace namespaces "nfs-test" not found, because `oc debug` creates its temporary Pod in the *current* project. Two fixes: `oc project default` (switch the context to a project that exists) or `oc debug node/ocp-node-1 --to-namespace=default` (used for every node capture in this book). Rule: after deleting a project, run `oc project` and move off it.

Add a real login: HTPasswd

OpenShift does not store users itself. It trusts an **identity provider** (LDAP/Active Directory, OpenID Connect, HTPasswd...). HTPasswd is a file of user names and hashed passwords kept in a Secret: perfect for a lab, and an EX280 objective.

```
# Reference: ~/ocp-install/add-htpasswd-admin.sh (prompts for the password, stores only a bcrypt hash)
htpasswd -c -B -b "$F" ocpadmin "$P1"
oc create secret generic htpass-secret --from-file=htpasswd="$F" -n openshift-config --dry-run=client -o
yaml | oc apply -f -
oc apply -f - <<'Y'
apiVersion: config.openshift.io/v1
kind: OAuth
metadata: {name: cluster}
spec:
  identityProviders:
  - name: lab-htpasswd
    mappingMethod: claim
    type: HTPasswd
    htpasswd: {fileData: {name: htpass-secret}}
Y
oc adm policy add-cluster-role-to-user cluster-admin ocpadmin
oc -n openshift-authentication rollout status deploy/oauth-openshift --timeout=300s
```

```
admin@bastion - real capture

[admin@bastion ~]$ oc get oauth cluster -o jsonpath="{range .spec.identityProviders[*]}{.name}{\"\t\"}{.
type}{\"\n\"}{end}"; oc get users; oc get identity
lab-htpasswd      HTPasswd
NAME              UID                                FULL NAME    IDENTITIES
ocpadmin          88970b2d-e4e2-49c8-ab06-a18dba58920a    lab-htpasswd:ocpadmin
NAME              IDP NAME      IDP USER NAME  USER NAME  USER UID
lab-htpasswd:ocpadmin  lab-htpasswd  ocpadmin       ocpadmin   88970b2d-e4e2-49c8-ab06-a18dba58920a
```

Three objects, three ideas:

OBJECT	WHAT IT IS	CREATED WHEN
OAuth <code>cluster</code> → identity provider <code>lab-htpasswd</code>	Where passwords are checked	When you apply the OAuth config
Identity <code>lab-htpasswd:ocpadmin</code>	"This login at this provider"	At the first successful login
User <code>ocpadmin</code>	The OpenShift account that RBAC refers to	At the first successful login (<code>mappingMethod: claim</code>)

NOTE

A user does not appear in `oc get users` until it has logged in once. If you grant a role before the first login, the binding is created anyway and starts working when the user appears.

```
admin@bastion — real capture

[admin@bastion ~]$ oc get clusterrolebinding -o wide | grep -E "NAME|ocpadmin"
NAME                                     ROLE
AGE   USERS                                GROUPS
SERVICEACCOUNTS
cluster-admin-0                         ClusterRole/cluster-admin
8h    ocpadmin
```

`oc adm policy add-cluster-role-to-user` created the ClusterRoleBinding `cluster-admin-0`: user `ocpadmin` holds the `cluster-admin` ClusterRole everywhere. In vSphere words: a permission with the *Administrator* role on the vCenter root, propagated.

Remove kubeadmin

Only after `ocpadmin` has logged in and proved it is cluster-admin:

```
# Reference
oc login -u ocpadmin https://api.ocp1.example.com:6443
oc auth can-i '*' '*' --all-namespaces # must print: yes
oc delete secret kubeadmin -n kube-system
```

```
admin@bastion — real capture

[admin@bastion ~]$ oc get secret kubeadmin -n kube-system
Error from server (NotFound): secrets "kubeadmin" not found
```

An error is the correct result here: the secret is gone, so the `kubeadmin` login no longer exists. The kubeconfig certificate still works as break-glass access.

SCREENSHOT S-03:

Web console login page showing the `lab-htpasswd` button (and no `kube:admin` button any more).

GOTCHA — REAL ISSUE

If you delete `kubeadmin` before another cluster-admin works, the only way back in is the installer's kubeconfig. If that file is also lost, you have lost the cluster. Keep `~/ocp1/auth/kubeconfig` backed up somewhere safe and offline.

RBAC in one table

NEED	COMMAND
Make someone admin of one project	<code>oc adm policy add-role-to-user admin alice -n mtv-test</code>
Read-only on one project	<code>oc adm policy add-role-to-user view bob -n mtv-test</code>
Cluster-wide admin	<code>oc adm policy add-cluster-role-to-user cluster-admin ocpadmin</code>
Group	<code>oc adm groups new vm-admins alice bob</code> then <code>oc adm policy add-role-to-group admin vm-admins -n mtv-test</code>
Can I...?	<code>oc auth can-i create virtualmachines -n mtv-test</code>
Can she...?	<code>oc auth can-i create virtualmachines -n mtv-test --as alice</code>
Stop everyone creating projects	<code>oc adm policy remove-cluster-role-from-group self-provisioner system:authenticated:oauth</code>

PRODUCTION

Use your company identity provider (Active Directory via LDAP, or Entra ID / Keycloak via OIDC), sync AD groups with `oc adm groups sync`, and grant roles to groups, never to individuals. OpenShift Virtualization adds the ClusterRoles `kubevirt.io:admin`, `kubevirt.io:edit` and `kubevirt.io:view`, aggregated into `admin`, `edit` and `view`, so project admins can manage VMs out of the box.

EXAM TIP

HTPasswd, users, groups, `oc adm policy`, removing `kubeadmin` and `self-provisioner` are core EX280 objectives. Practise the whole sequence in under ten minutes, including waiting for the `oauth-openshift` Pods to roll.

LAB — DO IT ON THE BASTION

Add a second htpasswd user `vmviewer` (extract the current file first with `oc extract secret/htpass-secret -n openshift-config --to=- > users.htpasswd`, add the user with `htpasswd -B -b`, then replace the secret). Grant `view` on `mtv-test`. Log in as `vmviewer` in a private browser window and confirm you can see the migrated VM but cannot start it.

Check yourself

Q1. You ran `oc get users` right after applying the OAuth config and `ocpadmin` is missing. Is the config broken? No. *Logic:* User and Identity objects are created on first login

(`mappingMethod: claim`). Log in once, then look again.

Q2. Why must you test `ocpadmin` before deleting the `kubeadmin` secret? Because `kubeadmin` is then gone for good. *Logic:* if the new identity provider is misconfigured, only the installer kubeconfig remains. Prove the replacement before removing the original.

Q3. `oc debug node/...` fails with "namespaces nfs-test not found", but you are cluster-admin. Why? Your current context points at a deleted project and `oc debug` creates its Pod there. *Logic:* permissions are fine; the target namespace does not exist. Use `oc project default` or `--to-namespace=default`.

Q4. Role vs ClusterRole, RoleBinding vs ClusterRoleBinding: which pair gives "admin in project mtv-test only"? The ClusterRole `admin` bound with a **RoleBinding** in `mtv-test`. *Logic:* the role defines *what*, the binding defines *where*. A RoleBinding limits even a ClusterRole to its own namespace.

Storage: NFS, CSI and Persistent Volumes

A VM is mostly its disks. On vSphere a disk is a VMDK file on a datastore. On OpenShift a VM disk is a **PersistentVolumeClaim (PVC)**: a request for storage that a **StorageClass** fulfils by creating a **PersistentVolume (PV)** through a **CSI driver**. Learn these four words and the rest of the book is easy.

OPENSIFT	VSPHERE	IN THIS LAB
StorageClass	A storage policy (SPBM) + the datastore it lands on	<code>nfs-csi</code>
CSI driver	The storage vendor's VASA provider / plug-in	<code>nfs.csi.k8s.io</code> (csi-driver-nfs v4.11.0)
PersistentVolume (PV)	The VMDK file	A folder on <code>/exports/ocp</code>
PersistentVolumeClaim (PVC)	The VM's "Hard disk 1" entry pointing at that VMDK	<code>mtv-test/test-ubuntu-vm-2275-dm9s9</code>
Access mode RWX	A shared datastore every host can open	Needed for live migration
Access mode RWO	A disk only one host can open at a time	Block arrays without RWX support

NOTE

RWX (ReadWriteMany) is the property that makes VM live migration possible: while a VM moves, the source and target nodes both have its disk open for a moment. It is the same reason vMotion without Storage vMotion needs a datastore shared by both hosts.

The NFS server (bastion side)

```
admin@bastion - real capture

[admin@bastion ~]$ sudo exportfs -v; showmount -e localhost
/exports/ocp 198.51.100.0/24(sync,no_wdelay,hide,no_subtree_check,sec=sys,rw,secure,no_root_squash,no_all_squash)
Export list for localhost:
/exports/ocp 198.51.100.0/24
```

OPTION	MEANING	WHY HERE
198.51.100.0/24	Who may mount	Covers the node VLAN (.32/27) and the bastion
rw	Read-write	VMs write to their disks
sync	Reply only after data is on disk	Safety for VM disks
no_root_squash	Root on the client stays root on the share	The CSI driver creates folders and sets ownership as root

```
# Reference: how the export was created
echo '/exports/ocp 198.51.100.0/24(rw, sync, no_root_squash, no_subtree_check)' | sudo tee /etc/exports.d/ocp.exports
sudo systemctl enable --now nfs-server
sudo firewall-cmd --permanent --add-service={nfs,mountd,rpc-bind} && sudo firewall-cmd --reload
sudo exportfs -ra
```

The CSI driver and StorageClass (cluster side)

```
# Reference: ~/ocp-install/install-nfs-csi.sh (key lines)
V=v4.11.0; B=https://raw.githubusercontent.com/kubernetes-csi/csi-driver-nfs/$V/deploy
oc apply -f $B/rbac-csi-nfs.yaml -f $B/csi-nfs-driverinfo.yaml
oc adm policy add-scc-to-user privileged -z csi-nfs-controller-sa -n kube-system
oc adm policy add-scc-to-user privileged -z csi-nfs-node-sa -n kube-system
oc apply -f $B/csi-nfs-controller.yaml -f $B/csi-nfs-node.yaml
oc apply -f sc-nfs-csi.yaml
```

NOTE

The two `add-scc-to-user privileged` lines are the OpenShift-specific part. A CSI node plug-in must mount filesystems on the host, which the default `restricted-v2` **Security Context Constraint** forbids. Upstream Kubernetes manifests do not know about SCCs, so on OpenShift you grant the driver's service accounts the `privileged` SCC yourself.

admin@bastion – real capture

```
[admin@bastion ~]$ oc get sc; oc get csidriver
NAME                PROVISIONER    RECLAIMPOLICY    VOLUMEBINDINGMODE    ALLOWVOLUMEEXPANSION    AGE
nfs-csi (default)   nfs.csi.k8s.io Delete          Immediate            true                    12h
NAME                ATTACHREQUIRED    PODINFOONMOUNT    STORAGECAPACITY    TOKENREQUESTS    REQUIRESREPUBLISH
MODES                AGE
nfs.csi.k8s.io      false             false             false              <unset>           false
Persistent          12h
```

```
admin@bastion - real capture

[admin@bastion ~]$ oc get sc nfs-csi -o yaml
allowVolumeExpansion: true
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
  name: nfs-csi
mountOptions:
- nfsvers=4.1
parameters:
  server: 198.51.100.24
  share: /exports/ocp
  subDir: ${pvc.metadata.namespace}-${pvc.metadata.name}-${pv.metadata.name}
provisioner: nfs.csi.k8s.io
reclaimPolicy: Delete
volumeBindingMode: Immediate
```

(Output trimmed: `uid` , `resourceVersion` and the last-applied annotation removed.)

FIELD	MEANING
<code>is-default-class: "true"</code>	PVCs that name no StorageClass get this one
<code>server</code> , <code>share</code>	The NFS export from the previous section
<code>subDir</code>	Each PV is a folder named <i>namespace-pvc-pv</i> , so you can find any VM's disk on the bastion
<code>reclaimPolicy: Delete</code>	Deleting the PVC deletes the folder. The disk is gone.
<code>volumeBindingMode: Immediate</code>	Create the PV as soon as the PVC appears, without waiting for a Pod
<code>allowVolumeExpansion: true</code>	You can grow a VM disk by editing the PVC size

```
admin@bastion - real capture

[admin@bastion ~]$ oc get pods -n kube-system -o wide | grep -E "NAME|csi-nfs"
NAME                                READY  STATUS   RESTARTS   AGE  IP              NODE      NOM
INATED  NODE  READINESS GATES
csi-nfs-controller-849b678558-dj4jp  5/5    Running  1 (8h ago)  8h   198.51.100.35   ocp-node-2
<none>      <none>
csi-nfs-node-jhcnx                  3/3    Running  0           12h   198.51.100.36   ocp-node-3
<none>      <none>
csi-nfs-node-smmxz                  3/3    Running  0           12h   198.51.100.34   ocp-node-1
<none>      <none>
csi-nfs-node-z8tfd                  3/3    Running  0           12h   198.51.100.35   ocp-node-2
<none>      <none>
```

One **controller** (creates and deletes folders) and one **node** Pod per node (a DaemonSet: mounts shares for the Pods on that node). Their IPs are node IPs because they use host networking.

Following a disk from PVC to folder

```
admin@bastion - real capture

[admin@bastion ~]$ oc get pv
NAME                                     CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM
STORAGECLASS   VOLUMEATTRIBUTESCLASS   REASON   AGE
pvc-21cff29a-6a67-4a87-be8c-1aab4e24f7a7   34144990004   RWX           Delete           Bound    openshift-virtualization-os-images/centos-stream10-041e2e2ceec9-12h
pvc-397a4657-b361-4c77-9370-52d6ef45892c   34144990004   RWX           Delete           Bound    openshift-virtualization-os-images/rhel10-c5b97492a6e3-12h
pvc-7d28af8c-864e-49c5-81be-30a007400184   34144990004   RWX           Delete           Bound    openshift-virtualization-os-images/rhel9-7005186c23b8-12h
pvc-97112ec8-28ce-4cb7-8936-e5f87cbdfbcc   34144990004   RWX           Delete           Bound    openshift-virtualization-os-images/fedora-1217dcc8c58d-12h
pvc-b501b26f-c8c4-4b40-b827-cd00be663a28   34144990004   RWX           Delete           Bound    openshift-virtualization-os-images/centos-stream9-e5d76202b136-12h
pvc-e18f93ca-a91e-46e9-9306-2bda130b4efd   11381663335   RWX           Delete           Bound    mtv-test-ubuntu-vm-2275-dm9s9-5h53m
pvc-e2adc166-a479-4558-a0b4-1cc1f3dcc299   34144990004   RWX           Delete           Bound    openshift-virtualization-os-images/rhel8-c6a08edc555b-12h
```

```
admin@bastion - real capture

[admin@bastion ~]$ sudo ls /exports/ocp | head -10
mtv-test-prime-eabb3341-7b20-4f01-8735-f6d440f95735-pvc-e18f93ca-a91e-46e9-9306-2bda130b4efd
openshift-virtualization-os-images-prime-741ce51b-0d56-4264-87c2-39060b16b6ca-pvc-7d28af8c-864e-49c5-81be-30a007400184
openshift-virtualization-os-images-prime-82e09ba6-2fc1-4442-9607-f453404b1530-pvc-b501b26f-c8c4-4b40-b827-cd00be663a28
openshift-virtualization-os-images-prime-8562c120-3d43-49a3-905d-486697b839e9-pvc-97112ec8-28ce-4cb7-8936-e5f87cbdfbcc
openshift-virtualization-os-images-prime-8f4e5e3c-65de-482d-92b3-1d76b43a696f-pvc-397a4657-b361-4c77-9370-52d6ef45892c
openshift-virtualization-os-images-prime-db63ec98-2b40-4e46-978e-0f1321a5d7a5-pvc-21cff29a-6a67-4a87-be8c-1aab4e24f7a7
openshift-virtualization-os-images-prime-fdc29b13-59e0-4737-ae09-ff13e2f17d40-pvc-e2adc166-a479-4558-a0b4-1cc1f3dcc299
```

Match the PV name to the folder: the migrated Ubuntu disk `pvc-e18f93ca-...` is the first folder. Its middle part says `prime-eabb3341-...` and not the PVC name: the disk was first written into a temporary "prime" PVC by the importer and then handed over (**rebound**) to the final PVC. That is how CDI (the Containerized Data Importer, Chapter 6) and MTV populate disks safely: the VM never sees a half-written disk.

TIP

`CAPACITY 11381663335` bytes is 10.6 GiB for a 10 GiB source disk. CDI adds room for filesystem overhead because on NFS the disk is a `disk.img` file inside a filesystem, not a raw block device.

GOTCHA — REAL ISSUE

OpenShift Virtualization could not guess the right access mode for an NFS StorageClass it had never seen, so new VM disks would have defaulted to RWO and lost live migration. The fix was to tell CDI about it through the **StorageProfile**: `oc patch storageprofile nfs-csi --type merge -p '{"spec":{"claimPropertySets":[{"accessModes":["ReadWriteMany"],"volumeMode":"Filesystem"}],"cloneStrategy":"copy"}}'`

PRODUCTION

A single NFS VM is a lab shortcut and a single point of failure. In production use the array's own CSI driver (NetApp Trident, Dell PowerStore/PowerMax CSI, Pure Portworx, IBM Block CSI, HPE CSI) or OpenShift Data Foundation, with **block** volumes (`volumeMode: Block`) and RWX where the array supports it (FC and iSCSI LUNs via the vendor CSI; NFS via Trident). Check that each StorageProfile reports RWX for live migration and supports snapshots for backup.

EXAM TIP

EX280: StorageClasses, PVCs, default class, access modes, expanding a PVC. EX316: why VM live migration needs RWX, StorageProfiles, DataVolumes.

LAB — DO IT ON THE BASTION

Create a 1 GiB PVC in a new project `storage-lab` with no StorageClass name, watch it bind (`oc get pvc -w`), find its folder under `/exports/ocp` , then delete the PVC and prove the folder disappears (`reclaimPolicy: Delete`). Delete the project and run `oc project default` afterwards (remember Chapter 4).

Check yourself

Q1. A PVC has been Pending for five minutes. Name three things to check, in order. `oc describe pvc` events; that a default StorageClass exists (`oc get sc`); that the CSI controller Pod is Running and can reach the NFS server. *Logic:* PVC events tell you which layer refused; the class decides which driver; the driver needs the backend.

Q2. You delete a migrated VM's PVC by mistake. Can you get the disk back from the bastion? No, not with this StorageClass. *Logic:* `reclaimPolicy: Delete` makes the CSI driver

delete the folder with the PV. Use `Retain` (or snapshots/backups) for data you cannot lose.

Q3. Why does the CSI driver need the `privileged` SCC on OpenShift but not on plain Kubernetes? OpenShift's default SCC forbids host mounts and privileged containers. *Logic:* SCCs are an OpenShift admission layer that upstream manifests do not know about, so you grant it to the driver's service accounts explicitly.

Q4. Why does a VM disk on NFS need RWX to live-migrate? During migration both the source and target virt-launcher Pods mount the same PVC at once. *Logic:* RWO allows one node; the target node could not attach the disk, so the migration is refused (`LiveMigratable=False`).

OpenShift Virtualization: VMs as Pods

OpenShift Virtualization is Red Hat's build of the upstream **KubeVirt** project. It runs a normal KVM virtual machine (the same hypervisor as RHEL and RHV) **inside a Pod**. The VM gets everything a Pod gets: scheduling, networking, storage, RBAC, monitoring, and the same `oc` commands.

VSPHERE	OPENSIFT VIRTUALIZATION	NOTE
ESXi host	Node with <code>/dev/kvm</code>	KVM is in the RHCOS kernel
VM (the <code>.vmx</code> definition)	<code>VirtualMachine</code> (VM) object	Exists even when powered off
Running VM process (<code>vmx</code>)	<code>VirtualMachineInstance</code> (VMI) + <code>virt-launcher-<vm></code> Pod	Exists only while running
vMotion	<code>VirtualMachineInstanceMigration</code> (VMIM)	Live migration, needs RWX disk
VMDK	PVC / DataVolume	Chapter 5
Port group	NetworkAttachmentDefinition (NAD)	Chapter 7
VM template / content library	Instance types + preferences + boot sources (DataSources)	Below
VMware Tools	QEMU guest agent + virtio drivers	MTV installs them during conversion
vCenter	The OpenShift API + web console <i>Virtualization</i> section	

Install: three operators

```
# Reference: ~/ocp-install/install-cnv-nmstate-mtv.sh (key part)
sub openshift-cnv      kubevirt-hyperconverged      stable      # Namespace + OperatorGroup + Subscription
sub openshift-nmstate  kubernetes-nmstate-operator  stable
sub openshift-mtv      mtv-operator                  release-v2.12
oc apply -f - <<'Y'
apiVersion: hco.kubevirt.io/v1beta1
kind: HyperConverged      # "turn Virtualization on" with defaults
metadata: {name: kubevirt-hyperconverged, namespace: openshift-cnv}
spec: {}
---
apiVersion: nmstate.io/v1
kind: NMState              # "turn NMState on"
metadata: {name: nmstate}
---
apiVersion: forklift.konveyor.io/v1beta1
kind: ForkliftController   # "turn MTV on"
metadata: {name: forklift-controller, namespace: openshift-mtv}
spec: {olm_managed: true}
Y
```

An **Operator** is installed in two steps: a **Subscription** tells OLM (the Operator Lifecycle Manager) which package and channel to follow; OLM installs a **ClusterServiceVersion (CSV)**, the operator itself. Then you create the operator's own **custom resource** (HyperConverged, NMState, ForkliftController) to say "deploy it".

admin@bastion – real capture

```
[admin@bastion ~]$ oc get subscription -A
NAMESPACE      NAME                                PACKAGE                                SOURCE                                CHANN
EL
openshift-cnv   kubevirt-hyperconverged            kubevirt-hyperconverged              redhat-operators                    stabl
e
openshift-mtv   mtv-operator                       mtv-operator                          redhat-operators                    relea
se-v2.12
openshift-nmstate  kubernetes-nmstate-operator        kubernetes-nmstate-operator          redhat-operators                    stabl
e
```

admin@bastion – real capture

```
[admin@bastion ~]$ oc get csv -A | grep -vE "^openshift-(operator-lifecycle|operators )" | awk '!seen
[\\$2]++' | head -20
NAMESPACE      NAME                                DISPLAY
VERSION        RELEASE  REPLACES                                PHASE
openshift-cnv   kubevirt-hyperconverged-operator.v4.22.9  OpenShift Virtu
alization      4.22.9    kubevirt-hyperconverged-operator.v4.22.
6 Succeeded
openshift-mtv   mtv-operator.v2.12.9                  Migration Toolk
it for Virtualization Operator  2.12.9    mtv-operator.v2.12.8
Succeeded
openshift-nmstate  kubernetes-nmstate-operator.4.22.0-202609230131  Kubernetes NMSt
ate Operator    4.22.0-202609230131
Succeeded
```

REPLACES ...v4.22.6 shows OLM already upgraded Virtualization from 4.22.6 to 4.22.9 by following the `stable` channel. `PHASE Succeeded` is the only healthy value.

SCREENSHOT S-04:

Operators > Installed Operators, all projects: OpenShift Virtualization 4.22.9, Migration Toolkit for Virtualization 2.12.9 and Kubernetes NMState Operator, each *Succeeded*.

```
admin@bastion - real capture
```

```
[admin@bastion ~]$ oc get hco -n openshift-cnv; oc get hco kubevirt-hyperconverged -n openshift-cnv -o jsonpath="{range .status.conditions[*]}{.type}={.status}{\\\"\\n\\\"}{end}"
```

NAME	AGE
kubevirt-hyperconverged	12h

ReconcileComplete=True
Available=True
Progressing=False
Degraded=False
Upgradeable=True

The **HyperConverged** (HCO) object is the one switch for all of Virtualization: KubeVirt, CDI (disk import), networking add-ons, SSP (templates), the console plug-in. `Available=True`, `Degraded=False` = healthy.

```
admin@bastion - real capture
```

```
[admin@bastion ~]$ oc get pods -n openshift-cnv | head -30
```

NAME	READY	STATUS	RESTARTS	AGE
aaq-operator-575758df8d-n2qcs	1/1	Running	0	8h
bridge-marker-m8dkr	1/1	Running	0	12h
...				
cdi-apiserver-7c7d7d56f5-wq5g2	1/1	Running	1 (12h ago)	12h
cdi-deployment-7cbbc6cbb-zxxtm	1/1	Running	0	8h
cdi-operator-7bb8d69cdb-bdtb6	1/1	Running	0	8h
cdi-uploadproxy-5bf887d9bc-xsxzt	1/1	Running	0	8h
cluster-network-addons-operator-84c86f8c9f-9kxlm	1/1	Running	0	8h
hco-operator-65668c9cb-vvgw6	1/1	Running	0	8h
...				
kube-cni-linux-bridge-plugin-czck8	1/1	Running	0	12h
...				
kubemacpool-mac-controller-manager-799884cb8f-dqcw2	1/1	Running	1 (12h ago)	12h
...				
kubevirt-console-plugin-6f89f54dbd-dtdmr	1/1	Running	0	12h
...				
ssp-operator-7fcf697f6d-4d7p9	1/1	Running	0	8h
virt-api-7c595746c7-5ptdw	1/1	Running	0	12h
virt-api-7c595746c7-j7l7f	1/1	Running	0	8h

POD FAMILY	JOB	VSPHERE ANALOGY
<code>virt-api</code> , <code>virt-controller</code> , <code>virt-handler</code> (one per node)	Create, schedule and run VMs	vpqd + hostd
<code>cdi-*</code>	Import, upload and clone disks	Content library / OVF import
<code>bridge-marker</code> , <code>kube-cni-linux-bridge-plugin</code>	Attach VMs to linux bridges	vSwitch / VDS
<code>kubemacpool-*</code>	Hand out unique MAC addresses	vCenter MAC allocation
<code>ssp-operator</code>	Templates, instance types, boot sources	VM templates
<code>kubevirt-console-plugin</code>	The <i>Virtualization</i> pages in the web console	vSphere Client plugin

Is hardware virtualization really there?

```

admin@bastion - real capture

[admin@bastion ~]$ oc get nodes -o custom-columns=NODE:.metadata.name,KVM:.status.allocatable.devices\kubevirt\io/kvm,CPU:.status.allocatable.cpu,MEM:.status.allocatable.memory
NODE          KVM   CPU    MEM
ocp-node-1    1k    11500m  48175496Ki
ocp-node-2    1k    11500m  48175492Ki
ocp-node-3    1k    11500m  48175504Ki

```

Every node advertises `devices.kubevirt.io/kvm: 1k` (1000 VM slots): `virt-handler` found `/dev/kvm`. A node without it would show `<none>` and no VM could be scheduled there. Allocatable CPU is 11.5 of 12 cores: OpenShift reserves the rest for the node itself.

```

admin@bastion - real capture

[admin@bastion ~]$ oc debug node/ocp-node-1 --to-namespace=default -q -- chroot /host bash -c "ls -l /dev/kvm; grep -c -E \"vmx|svm\" /proc/cpuinfo"
crw-rw-rw-. 1 root kvm 10, 232 Oct  5 20:27 /dev/kvm
24

```

`oc debug node/...` starts a temporary privileged Pod on the node and `chroot /host` puts you in the node's real filesystem: the supported way to "SSH" into RHCOS. `/dev/kvm` exists, and 24 lines of `/proc/cpuinfo` carry the Intel `vmx` flag (two flag fields for each of the 12 vCPUs), proving vSphere's *Expose hardware assisted virtualization* is on.

GOTCHA — REAL ISSUE

Nested virtualization works, but it is slow and VMware does not support it for production. A nested VM's disk and CPU-heavy work (like the virt-v2v conversion in Chapter 9) run several times slower than on bare metal.

Boot sources, instance types and preferences

Virtualization downloads ready-to-boot OS images and keeps them up to date. They are the new "golden templates".

```
admin@bastion — real capture

[admin@bastion ~]$ oc get datasource -n openshift-virtualization-os-images
NAME                                AGE
centos-stream10                     12h
centos-stream9                      12h
fedora                              12h
rhel10                              12h
rhel7                               12h
rhel8                               12h
rhel9                               12h
win10                               12h
win11                               12h
win2k16                             12h
win2k19                             12h
win2k22                             12h
win2k25                             12h
```

```
admin@bastion — real capture

[admin@bastion ~]$ oc get dv -A | head -15
NAMESPACE                                NAME                                PHASE      PROGRESS  RESTARTS  AG
E
openshift-virtualization-os-images      centos-stream10-041e2e2ceec9      Succeeded  100.0%    12
h
openshift-virtualization-os-images      centos-stream9-e5d76202b136      Succeeded  100.0%    12
h
openshift-virtualization-os-images      fedora-1217dcc8c58d              Succeeded  100.0%    12
h
openshift-virtualization-os-images      rhel10-c5b97492a6e3              Succeeded  100.0%    12
h
openshift-virtualization-os-images      rhel8-c6a08edc555b              Succeeded  100.0%    12
h
openshift-virtualization-os-images      rhel9-7005186c23b8              Succeeded  100.0%    12
h
```

Thirteen **DataSources** exist but only six have a **DataVolume** behind them: the six Linux images Red Hat can ship automatically. `rhel7` and all Windows entries are empty pointers, because Microsoft licensing means you upload your own Windows image. A **DataVolume (DV)** is "a PVC plus instructions for filling it" (import from a URL or registry, upload, or clone); CDI does the work and the DV reports `PHASE` and `PROGRESS`.

```
admin@bastion - real capture

[admin@bastion ~]$ oc get virtualmachineclusterinstancetype | head -12; echo; oc get virtualmachineclusterpreference | grep -E "NAME|rhel|ubuntu|windows.2k22|fedora"
NAME          AGE
cx1.2xlarge   12h
cx1.2xlarge1gi 12h
cx1.4xlarge   12h
...
cx1.large     12h
cx1.medium    12h
cx1.xlarge    12h

NAME          AGE
fedora        12h
...
rhel.10       12h
rhel.8        12h
rhel.9        12h
rhel.9.desktop 12h
...
ubuntu        12h
windows.2k22  12h
windows.2k22.virtio 12h
```

CONCEPT	ANSWERS	EXAMPLE
Instance type	How big? vCPU and memory	u1.medium = 1 vCPU, 4 GiB; cx1.* = compute-exclusive (dedicated CPUs)
Preference	What OS? best devices for it	rhel.9 : virtio disks and NICs; windows.2k22 : Hyper-V enlightenments, TPM
DataSource	Which disk image?	rhel9

Instance type + preference + DataSource = a new VM in three clicks. They replace the older OpenShift **Templates** (still present: `oc get templates -n openshift | grep rhel9`).

Creating, starting and stopping a VM

SCREENSHOT S-05:

Virtualization > Catalog > InstanceTypes: the RHEL 9 boot source tile selected, series U, size medium, project default .

```
# Reference: the same VM from YAML (oc apply -f rhel9-demo.yaml)
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata: {name: rhel9-demo, namespace: default}
spec:
  runStrategy: Always # Always | Halted | Manual | RerunOnFailure
 instancetype: {name: u1.medium}
  preference: {name: rhel.9}
  dataVolumeTemplates:
  - metadata: {name: rhel9-demo-root}
    spec:
      sourceRef: {kind: DataSource, name: rhel9, namespace: openshift-virtualization-os-images}
      storage: {resources: {requests: {storage: 30Gi}}}}
  template:
    spec:
      domain: {devices: {}}
      volumes:
      - {name: rootdisk, dataVolume: {name: rhel9-demo-root}}
      - name: cloudinitdisk
        cloudInitNoCloud: {userData: "#cloud-config\nuser: cloud-user\npassword: <PASSWORD>\nchpasswd:
{expire: false}"}
```

NEED	WEB CONSOLE	CLI
Start	VM > Actions > Start	<code>virtctl start rhel9-demo</code> or <code>oc patch vm rhel9-demo --type merge -p '{"spec":{"runStrategy":"Always"}}'</code>
Stop	Actions > Stop	<code>virtctl stop rhel9-demo</code> or <code>runStrategy: Halted</code>
Console	<i>Console</i> tab (VNC or serial)	<code>virtctl console rhel9-demo</code> / <code>virtctl vnc rhel9-demo</code>
Live migrate	Actions > Migrate	<code>virtctl migrate rhel9-demo</code>
Where is it running?	<i>Overview</i> , Node field	<code>oc get vmi rhel9-demo -o wide</code>
Snapshot	<i>Snapshots</i> tab	<code>oc create -f vmsnapshot.yaml</code> (<code>VirtualMachineSnapshot</code>)

admin@bastion — real capture

```
[admin@bastion ~]$ which virtctl || ls ~/bin
which: no virtctl in (/home/admin/bin:/home/admin/.local/bin:/home/admin/bin:/usr/local/bin:/usr/bin:/usr/local/sbin:/usr/sbin)
kubectl
oc
openshift-install
```

GOTCHA — REAL ISSUE

`virtctl` is not installed on the bastion yet, so every VM action in this lab was done in the web console or with `oc patch ... runStrategy`. Download it from the cluster itself: web console ? menu > *Command Line Tools* > *virtctl* (Linux x86_64) (the Pod `hyperconverged-cluster-cli-download-...` in `openshift-cnv` serves that download). Put it in `~/bin`. The exams expect you to use it.

The API you are now using

```
admin@bastion - real capture

[admin@bastion ~]$ oc api-resources --api-group=kubevirt.io
NAME                                SHORNNAMES  APIVERSION  NAMESPACED  KIND
kubevirts                          kv,kvs      kubevirt.io/v1  true        KubeVirt
virtualmachineinstancemigrations  vmim,vmims  kubevirt.io/v1  true        VirtualMachineI
nstanceMigration
virtualmachineinstancepresets      vmipreset,vmipresets  kubevirt.io/v1  true        VirtualMachineI
nstancePreset
virtualmachineinstancereplicaset  vmirs,vmirss  kubevirt.io/v1  true        VirtualMachineI
nstanceReplicaSet
virtualmachineinstances            vmi,vmis     kubevirt.io/v1  true        VirtualMachineI
nstance
virtualmachines                    vm,vms       kubevirt.io/v1  true        VirtualMachine
```

Learn the short names: `oc get vm`, `oc get vmi`, `oc get vmim`. Use `oc explain vm.spec.runStrategy` (or any field path) whenever you forget a field: it is the built-in reference, and it is allowed in the exam.

PRODUCTION

Turn on the HyperConverged features you need deliberately (`oc edit hco -n openshift-cnv`): live-migration limits (`liveMigrationConfig.parallelMigrationsPerCluster`), a dedicated migration network, CPU model for mixed hardware, and the `HostPathProvisioner` only if you really want node-local disks. Set **eviction strategy** `LiveMigrate` so node drains (upgrades) move VMs instead of stopping them.

EXAM TIP

EX316 (Red Hat Certified Specialist in OpenShift Virtualization) covers: installing the operator, creating VMs from instance types and templates, VM disks and DataVolumes, VM networks (NAD, bridge), live migration and node maintenance, snapshots, cloning, importing VMs, and RBAC for VMs. Every chapter from here maps to it.

LAB — DO IT ON THE BASTION

(1) Install `virtctl` as described. (2) Create `rhel9-demo` from the YAML above in a project `virt-lab`. (3) Watch `oc get dv,vm,vmi,pods -n virt-lab -w` and write down the order objects appear. (4) Live-migrate it with `virtctl migrate rhel9-demo -n virt-lab` and watch `oc get vmim -n virt-lab` and the node change in `oc get vmi -o wide`. (5) Delete the project and run `oc project default`.

Check yourself

Q1. `oc get vm` shows `ubuntu-noble-24.04-cloudimg` but `oc get vmi` shows nothing. Is the VM broken? No, it is powered off. *Logic:* the VM object is the definition (like the `.vmx`); a VMI and its `virt-launcher` Pod exist only while it runs.

Q2. A node shows `<none>` in the KVM column. What will happen to a VM scheduled there, and what do you check in vSphere? It will not be scheduled there (`Insufficient devices.kubevirt.io/kvm`). *Logic:* `virt-handler` only advertises KVM when `/dev/kvm` exists; on vSphere tick *Expose hardware assisted virtualization* on the node VM (powered off).

Q3. Why do `win2k22` and `rhel7` `DataSources` exist but have no `DataVolume`? No image is imported for them automatically (Windows because of Microsoft licensing; RHEL 7 is not in the automatic boot-source list on this release). *Logic:* the DataSource is a pointer; you upload your own image (for example with `virtctl image-upload`) and point it there.

Q4. What is the difference between `runStrategy: Always` and `RerunOnFailure`? `Always` restarts the VM whenever it stops, even after a guest shutdown; `RerunOnFailure` restarts only after a failure, so a clean shutdown from inside the guest stays off. *Logic:* it is the VM equivalent of vSphere HA restart vs "leave powered off".

Q5. What is the supported way to get a shell on an RHCOS node? `oc debug node/<name>` then `chroot /host`. *Logic:* RHCOS is managed by the Machine Config Operator; SSH edits are not tracked and are overwritten, so access goes through the API and is audited.

VM Networking: NMState, Bridges and VLANs

Pods live on a private overlay network (the `clusterNetwork` 10.128.0.0/14 from Chapter 3) and reach the outside through NAT and Routes. A migrated server VM usually needs something different: **its own IP on the same VLAN it had in VMware**, so clients, firewalls and DNS keep working. That needs three layers, and each one maps to something you already build in vSphere.

LAYER	OPENSIFT OBJECT	VSPHERE EQUIVALENT	IN THIS LAB
Physical uplink	Node NIC <code>ens34</code>	vmnic on the ESXi host	NIC2 of each node VM, on a trunk port group
Switch	Linux bridge <code>br-vm</code> , created by a NodeNetworkConfigurationPolicy (NNCP)	Standard / distributed vSwitch	VLAN-aware, trunk 2 to 4094
VM network	NetworkAttachmentDefinition (NAD)	Port group with a VLAN ID	<code>vlan-110</code> , <code>vlan-120</code> , <code>vlan-130</code>

```
VM (net-0) → NAD default/vlan-110 (VLAN 110) → br-vm (linux bridge, trunk) → ens34 → vSphere trunk PG (VLAN 4095) → physical switch
```

NOTE

Because the OpenShift nodes are themselves VMware VMs, NIC2 sits on a vSphere port group with **VLAN 4095** (VGT, "pass all VLAN tags to the guest"), and that port group must allow *Promiscuous mode*, *MAC address changes* and *Forged transmits*. Without that, ESXi drops every frame from a nested VM, because its MAC is not the node VM's MAC. On bare metal this whole note disappears: `ens34` is a real NIC on a real trunk.

```
# Reference: New-OcpVmTrunk.ps1 (jump host) creates the trunk port group and moves NIC2 of OCP-NODE-1/2/3 onto it
PS C:\> .\New-OcpVmTrunk.ps1 -VCenter vcenter.example.com
```

Layer 2: the bridge (NMState)

The **Kubernetes NMState Operator** lets you configure node networking (bonds, VLANs, bridges, IPs, routes, DNS) through the API instead of editing nodes. You declare the *desired state*; a handler Pod on each node applies it, tests it, and **rolls it back automatically** if the node loses connectivity to the API.

```
# Reference: ~/ocp-install/vm-networks.sh, part 1
apiVersion: nmstate.io/v1
kind: NodeNetworkConfigurationPolicy
metadata: {name: br-vm-ens34}
spec:
  nodeSelector: {node-role.kubernetes.io/worker: ""}
  desiredState:
    interfaces:
      - name: br-vm
        type: linux-bridge
        state: up
        ipv4: {enabled: false}           # the bridge carries VM traffic only; the node has no IP on it
        ipv6: {enabled: false}
        bridge:
          options: {stp: {enabled: false}}
          port:
            - name: ens34
              vlan:
                mode: trunk
                trunk-tags:
                  - id-range: {min: 2, max: 4094}
```

```
admin@bastion - real capture

[admin@bastion ~]$ oc get nncp; oc get nnce
NAME          STATUS    REASON
br-vm-ens34   Available SuccessfullyConfigured
NAME          STATUS    STATUS AGE   REASON
ocp-node-1.br-vm-ens34   Available   8h         SuccessfullyConfigured
ocp-node-2.br-vm-ens34   Available   8h         SuccessfullyConfigured
ocp-node-3.br-vm-ens34   Available   8h         SuccessfullyConfigured
```

The **policy** (NNCP) is the cluster-wide wish; there is one **enactment** (NNCE) per node it applied to, each with its own status. If one node fails, its NNCE says **Failing** with the reason, and the others are untouched.

TIP

`nodeSelector: node-role.kubernetes.io/worker: ""` matched all three nodes because on a compact cluster every node carries the `worker` role label. On a cluster with separate workers, the same policy would deliberately skip the control plane.

```

admin@bastion — real capture

[admin@bastion ~]$ oc get nns ocp-node-1 -o jsonpath="{range .status.currentState.interfaces[*]}{.name}{\n\t\t}{.type}{\n\t\t}{.state}{\n\n}{end}" | grep -vE "veth|genev|ovs-interface" | head -20
br-ex    ovs-bridge    up
br-vm    linux-bridge    up
ens33    ethernet       up
ens34    ethernet       up
lo       loopback        up

```

The **NodeNetworkState** (NNS) is NMState's live report of each node. You can read it here, without logging in to the node:

- `ens33` is NIC1. Its IP (.34) lives on `br-ex`, the Open vSwitch bridge OVN-Kubernetes builds for the cluster network. That is why `ens33` shows no IP on the node console: normal, not a fault.
- `ens34` is NIC2, enslaved to `br-vm`, our linux bridge.

```

admin@bastion — real capture

[admin@bastion ~]$ oc debug node/ocp-node-1 --to-namespace=default -q -- chroot /host bash -c "ip -br link show master br-vm; bridge vlan show dev ens34 | head -4"
ens34      UP                00:50:56:aa:00:12 <BROADCAST,MULTICAST,UP,LOWER_UP>
port       vlan-id
ens34      1 PVID Egress Untagged
           2
           3

```

From inside the node: `ens34` (MAC `00:50:56:aa:00:12`, the same MAC the agent config called `ens224` in Chapter 3) is the only port of `br-vm`, and it accepts VLANs 2, 3 and onward (trimmed with `head`; the list runs to 4094). VLAN 1 is the untagged native VLAN.

Layer 3: the VM networks (NADs)

```

# Reference: ~/ocp-install/vm-networks.sh, part 2 (one of three)
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: vlan-110
  namespace: default
  annotations: {k8s.v1.cni.cncf.io/resourceName: bridge.network.kubevirt.io/br-vm}
spec:
  config: '{"cniVersion":"0.3.1","name":"vlan-110","type":"bridge","bridge":"br-vm","vlan":110,"macspoofchk":true,"ipam":{"}}'

```

```
admin@bastion — real capture

[admin@bastion ~]$ oc get net-attach-def -A
NAMESPACE          NAME          AGE
default             vlan-120      8h
default             vlan-130      8h
default             vlan-110      8h
openshift-ovn-kubernetes default       13h
```

```
admin@bastion — real capture

[admin@bastion ~]$ oc get net-attach-def vlan-110 -n default -o jsonpath="{.spec.config}" | python3 -m json.tool
{
  "cniVersion": "0.3.1",
  "name": "vlan-110",
  "type": "bridge",
  "bridge": "br-vm",
  "vlan": 110,
  "macspoofchk": true,
  "ipam": {}
}
```

KEY	MEANING
"type": "bridge"	Use the linux-bridge CNI plug-in
"bridge": "br-vm"	Plug into the bridge the NNCP created
"vlan": 110	Tag this VM's traffic with VLAN 110: the "VLAN ID" field of a port group
"macspoofchk": true	Drop frames whose source MAC is not the VM's own (like <i>Forged transmits: Reject</i>)
"ipam": {}	OpenShift gives the VM no IP; the guest uses DHCP or its static IP, as on VMware
resourceName annotation	Only schedule VMs on nodes that actually have br-vm

NOTE

NADs in the default namespace can be used by VMs in **any** project (reference it as default/vlan-110). A NAD in a project is visible only to that project. Put shared, admin-controlled networks in default (or a dedicated namespace) and keep project NADs for project-specific needs.

SCREENSHOT S-06:

Networking > NodeNetworkConfigurationPolicy: `br-vm-ens34` Available on 3 nodes, with the topology view showing `ens34` under `br-vm`.

SCREENSHOT S-07:

Networking > NetworkAttachmentDefinitions, project `default`: `vlan-110`, `vlan-120`, `vlan-130`, type *Linux bridge*.

GOTCHA — REAL ISSUE

Applying the NNCP is a **cluster networking change** on every node. NMState will roll it back if a node loses its API connection, but a mistake on the wrong NIC (for example `ens33`, which carries the node IP) can still take a node off the network for the rollback timeout. Always name the spare NIC, check `oc get nns <node>` first, and apply to one node (`nodeSelector: kubernetes.io/hostname: ocp-node-1`) before all.

GOTCHA — REAL ISSUE

The VLAN ID 110 for the old network was *assumed* from the port-group name, not confirmed with the network team. If a migrated VM on `vlan-110` gets no traffic, compare the real VLAN ID on the vSphere port group (*Edit settings > VLAN*) with the NAD's "vlan" value first.

PRODUCTION

Use a **bond** of two NICs under the bridge (NNCP `type: bond`, `mode: 802.3ad` or `active-backup`), one NAD per production VLAN, and consider OVN-Kubernetes **localnet** secondary networks (`type: ovn-k8s-cni-overlay`, `topology: localnet`) if you want network policy on VM traffic. Keep live-migration traffic on its own network (HCO `liveMigrationConfig.network`).

EXAM TIP

EX316: create an NNCP for a linux bridge, create a NAD (YAML and console), attach a VM to it as a second NIC, and explain pod network (masquerade) vs bridge (L2) binding.

LAB — DO IT ON THE BASTION

(1) Read the NNS of all three nodes and find the MAC of `ens34` on each. Compare with `agent-config.yaml`. (2) Add a second NIC on `default/vlan-120` to `rhel9-demo` from Chapter 6 (console: *Configuration > Network > Add network interface*), boot it, and run `ip a` in the guest. (3) Explain why it gets no IP if VLAN 120 has no DHCP server.

Check yourself

Q1. A NAD says "vlan": 110 . Where does the VLAN tag get added: in the guest, on br-vm , or on the physical switch? On br-vm , at the VM's bridge port. *Logic:* the guest sends untagged frames (like a VM on a port group with VLAN 110); the bridge tags them and the trunk on ens34 carries them out tagged.

Q2. The NNCP shows Available but a VM on vlan-110 cannot reach its gateway. In this nested lab, what two vSphere settings do you check? That NIC2 is on the trunk port group (VLAN 4095) and that the port group allows promiscuous mode, MAC changes and forged transmits. *Logic:* the bridge is fine (NNCE says so); frames are being dropped outside OpenShift, by the ESXi vSwitch.

Q3. Why does ens33 have no IP address when you look at the node console? OVN-Kubernetes moves the node IP onto the OVS bridge br-ex , with ens33 as its port. *Logic:* the bridge must own the IP so it can switch Pod and node traffic on the same uplink.

Q4. What is the difference between an NNCP and a NAD? The NNCP configures the node (creates br-vm on ens34); the NAD defines a network VMs can attach to (VLAN 110 on br-vm). *Logic:* one is the vSwitch, the other is the port group.

Migration Toolkit for Virtualization (MTV)

MTV (upstream project *Forklift*) is the OpenShift operator that moves VMs from vSphere, Red Hat Virtualization, OpenStack, Hyper-V, OVA files or another OpenShift cluster into OpenShift Virtualization. It reads the source VM, copies its disks into PVCs, converts the guest (removes VMware Tools, adds virtio drivers and the QEMU guest agent, fixes the bootloader) with **virt-v2v**, and creates a `VirtualMachine` with the same CPU, memory, MACs and disks.

MTV never changes or deletes the source VM in a cold migration. It only reads it. That makes the first migration of any VM safe to try.

The objects, in the order you create them

ORDER	MTV OBJECT	MEANING	IN THIS LAB
1	Provider (source)	A connection to vCenter: URL, credentials Secret, optional VDDK image	<code>vcenter-lab</code>
(auto)	Provider (destination)	This OpenShift cluster	<code>host</code>
2	NetworkMap	Source port group → Pod network or a NAD	<code>vcenter-lab-net : 110 → default/vlan-110</code>
3	StorageMap	Source datastore → StorageClass + access mode + volume mode	<code>vcenter-lab-storage : DATASTORE-01 → nfs-csi RWX Filesystem</code>
4	Plan	Which VMs, which maps, target project, cold or warm	<code>test-ubuntu</code>
5	Migration	One run of a plan (the <i>Start</i> button)	<code>test-ubuntu-1</code>

The operator

```
admin@bastion — real capture

[admin@bastion ~]$ oc get csv -n openshift-mtv
NAME                                     DISPLAY                                     VERSION  RELEASE  REPLACES
PHASE
mtv-operator.v2.12.9  Migration Toolkit for Virtualization Operator  2.12.9           mtv-operator.v2.12.8  Succeeded
```

```
admin@bastion — real capture

[admin@bastion ~]$ oc get forkliftcontroller -n openshift-mtv; oc get pods -n openshift-mtv
NAME          AGE
forklift-controller  12h

NAME          READY  STATUS    RESTARTS  AGE
forklift-api-97fbd7cf-bzwh2  1/1    Running    0          12h
forklift-cli-download-77d8546b5f-d9sxh  1/1    Running    0          12h
forklift-controller-5f6b76dd7c-d9l8z    2/2    Running    0          8h
forklift-operator-845fbf655b-47547      1/1    Running    0          8h
forklift-ova-proxy-7f6dd84f44-pdtln     1/1    Running    0          8h
forklift-ui-plugin-7d8fb5df84-kt6xj     1/1    Running    0          8h
forklift-validation-7bf5749c7b-lng6x    1/1    Running    0          12h
forklift-volume-populator-controller-685dc5768f-2mz58  1/1    Running    0          8h
```

POD	JOB
forklift-controller (2 containers)	The brain: runs plans; its <code>inventory</code> container caches the vCenter inventory
forklift-validation	Checks each VM against rules (unsupported disks, snapshots, missing CBT...) and shows warnings on the plan
forklift-api	Webhooks that reject invalid objects
forklift-ui-plugin	The <i>Migration for Virtualization</i> pages in the web console
forklift-volume-populator-controller	Fills PVCs for copy-offload and other populators
forklift-cli-download	Serves the <code>kubectl-mtv</code> CLI plug-in

```
admin@bastion — real capture

[admin@bastion ~]$ oc api-resources --api-group=forklift.konveyor.io
NAME                                SHORTNAMES  APIVERSION                                NAMESPACE  KIND
conversions                        forklift.konveyor.io/v1beta1  true        Conversion
forkliftcontrollers                forklift.konveyor.io/v1beta1  true        ForkliftControll
er
hooks                              forklift.konveyor.io/v1beta1  true        Hook
hosts                              forklift.konveyor.io/v1beta1  true        Host
hypervproviderservers              forklift.konveyor.io/v1beta1  true        HyperVProviderSe
rver
migrations                        forklift.konveyor.io/v1beta1  true        Migration
networkmaps                       forklift.konveyor.io/v1beta1  true        NetworkMap
openstackvolumeopulators           osv,osvps  forklift.konveyor.io/v1beta1  true        OpenstackVolumeP
opulator
ovaproviderservers                 forklift.konveyor.io/v1beta1  true        OVAProviderServe
r
ovirtvolumeopulators               ovvp,ovvps forklift.konveyor.io/v1beta1  true        OvirtVolumePopul
ator
plans                              forklift.konveyor.io/v1beta1  true        Plan
providers                          forklift.konveyor.io/v1beta1  true        Provider
storagemaps                       forklift.konveyor.io/v1beta1  true        StorageMap
vspherexcopyvolumeopulators        vxvp,vxvps forklift.konveyor.io/v1beta1  true        VSphereXcopyVolu
mePopulator
```

Two newer kinds worth knowing: **Hook** runs an Ansible playbook or script before or after a VM migrates (for example, stop an application cleanly), and **VSphereXcopyVolumePopulator** is *storage copy offload*: when the source datastore and the target StorageClass are on the same array, the array copies the disk itself (XCOPY) instead of sending it over the network.

Step 1 · A vCenter account for MTV

MTV logs in to vCenter, reads the inventory, reads disks, takes snapshots (warm migration) and powers the source VM off at cutover. Give it its own account with exactly those privileges.

```
# Reference: jump host, PowerCLI. Creates role MTV-Migration (18 privileges), SSO user mtv@vsphere.local,
# and a propagated permission on the vCenter root. Prompts for passwords, writes nothing to disk.
PS C:\> .\New-MtvVcenterAccount.ps1 -Vcenter vcenter.example.com -SkipCertificateCheck
```

PRIVILEGE GROUP	PRIVILEGES	WHY
Sessions	Validate session	Keep the API session alive
Datastore	Browse, Low level file operations	Find and read VMDKs
VM > Interaction	Power off, Power on	Shut down the source at cutover, power it on for rollback
VM > Guest operations	Query, Execute, Modify	Read the guest IP configuration (preserve static IPs)
VM > Provisioning	Allow disk access, read-only disk access, file access, VM download, file upload, Clone, Mark as template	Read disks through VDDK/NFC
VM > Snapshot management	Create, Remove	Consistent reads, CBT deltas
VM > Change configuration	Toggle disk change tracking	Turn on CBT for warm migration

SCREENSHOT S-08:

vSphere Client, *Administration > Roles*: MTV-Migration with its privileges, and the *Permissions* tab of vCenter showing `mtv@vsphere.local` propagated.

Step 2 · The source provider

```
# Reference: ~/ocp-install/add-mtv-vsphere-provider.sh (prompts for the mtv@vsphere.local password)
oc create secret generic vcenter-lab -n openshift-mtv \
  --from-literal=user=mtv@vsphere.local --from-literal=password="$PW" \
  --from-literal=url=https://vcenter.example.com/sdk --from-literal=insecureSkipVerify=true \
  --dry-run=client -o yaml \
  | oc label -f - --local -o yaml createdForResourceType=providers createdForProviderType=vsphere \
  | oc apply -f -
oc apply -f - <<'Y'
apiVersion: forklift.konveyor.io/v1beta1
kind: Provider
metadata: {name: vcenter-lab, namespace: openshift-mtv}
spec:
  type: vsphere
  url: https://vcenter.example.com/sdk
  secret: {name: vcenter-lab, namespace: openshift-mtv}
  settings: {sdkEndpoint: vcenter}
Y
```

GOTCHA — REAL ISSUE

The first attempt failed in MTV's admission webhook: the Secret carried `createdForResourceType=providers` but not `createdForProviderType=vsphere`, and MTV rejects one without the other. Both labels must arrive in the same request, which is why the script adds them with `oc label --local` before `oc apply`.

admin@bastion — real capture

```
[admin@bastion ~]$ oc get providers -n openshift-mtv
```

NAME	TYPE	STATUS	READY	CONNECTED	INVENTORY	URL
AGE						
host	openshift	Ready	True	True	True	
12h						
vcenter-lab	vsphere	Ready	True	True	True	https://vcenter.example.com/sdk
9m						6h2

```

admin@bastion — real capture

[admin@bastion ~]$ oc get provider vcenter-lab -n openshift-mtv -o yaml | sed -n "/^spec:/,\$p" | grep -
vE "lastTransitionTime|observedGeneration" | head -60
spec:
  secret:
    name: vcenter-lab
    namespace: openshift-mtv
  settings:
    sdkEndpoint: vcenter
  type: vsphere
  url: https://vcenter.example.com/sdk
status:
  conditions:
  - category: Warn
    message: TLS is susceptible to machine-in-the-middle attacks when certificate
      verification is skipped.
    reason: SkipTLSVerification
    status: "True"
    type: ConnectionInsecure
  - category: Required
    message: Connection test, succeeded.
    reason: Tested
    status: "True"
    type: ConnectionTestSucceeded
  - category: Advisory
    message: Validation has been completed.
    reason: Completed
    status: "True"
    type: Validated
  - category: Required
    message: The inventory has been loaded.
    reason: Completed
    status: "True"
    type: InventoryCreated
  - category: Required
    message: The provider is ready.
    status: "True"
    type: Ready
  fingerprint: AA:BB:CC:DD:EE:FF:00:11:22:33:44:55:66:77:88:99:AA:BB:CC:DD
  phase: Ready
  secretResourceVersion: "386243"

```

How to read MTV conditions (the same pattern applies to maps and plans):

- **category: Required** conditions must all be **True** for the object to work: here *ConnectionTestSucceeded*, *InventoryCreated* and *Ready*.
- **Warn** and **Advisory** are information. *ConnectionInsecure* is the honest price of `insecureSkipVerify=true`.
- The **fingerprint** is the SHA-1 of vCenter's certificate. In production you paste the vCenter CA instead of skipping verification.
- There is **no** **vddkInitImage** under **settings**. This single missing line is why the test migration in Chapter 9 took 84 minutes for 10 GiB.

SCREENSHOT S-09:

Migration for Virtualization > Providers: `host` and `vcenter-lab` both Ready, with the vcenter-lab inventory counts (VMs, hosts, networks, datastores).

VDDK: the line that makes it fast

VDDK is VMware's disk library, the same one backup products use. Red Hat may not redistribute it, so you download it from Broadcom and wrap it in a tiny image. With it, disk data comes straight from the ESXi host over NBD (port 902); without it, virt-v2v reads the disk over HTTPS through vCenter.

```
# Reference: build the VDDK image (bastion, podman), then point the provider at it
tar -xzf ~/VMware-vix-disklib-8.0.*.x86_64.tar.gz -C ~/vddk
cat > ~/vddk/Containerfile <<'EOF'
FROM registry.access.redhat.com/ubi9/ubi-minimal
USER 1001
COPY vmware-vix-disklib-distrib /vmware-vix-disklib-distrib
RUN mkdir -p /opt
ENTRYPOINT ["cp", "-r", "/vmware-vix-disklib-distrib", "/opt"]
EOF
podman build -t <REGISTRY>/openshift-mtv/vddk:8.0 ~/vddk && podman push <REGISTRY>/openshift-mtv/vddk:8.0
0
oc patch provider vcenter-lab -n openshift-mtv --type merge \
  -p '{"spec":{"settings":{"vddkInitImage":"<REGISTRY>/openshift-mtv/vddk:8.0"}}}'
```

Step 3 · Network and storage maps

admin@bastion – real capture

```
[admin@bastion ~]$ oc get networkmap,storagemap -n openshift-mtv
```

NAME	READY	AGE
networkmap.forklift.konveyor.io/vcenter-lab-net	True	5h54m

NAME	READY	AGE
storagemap.forklift.konveyor.io/vcenter-lab-storage	True	5h54m

```
[admin@bastion ~]$ oc get networkmap vcenter-lab-net -n openshift-mtv -o yaml | sed -n "/^spec:/,\$p" |  
grep -vE "lastTransitionTime|observedGeneration"  
spec:  
  map:  
  - destination:  
      name: vlan-110  
      namespace: default  
      type: multus  
    source:  
      name: "VM-Network-110"  
  provider:  
    destination:  
      name: host  
      namespace: openshift-mtv  
    source:  
      name: vcenter-lab  
      namespace: openshift-mtv  
status:  
  conditions:  
  - category: Required  
    message: The network map is ready.  
    status: "True"  
    type: Ready  
  references:  
  - id: network-1044  
    name: "VM-Network-110"
```

`destination type: multus` = "a NAD" (Multus is the CNI that attaches secondary networks); the other choice, `type: pod`, would put the VM on the Pod network behind NAT. `references` shows MTV resolved the name `"110"` to vCenter's managed object ID `network-1044`. The quotes around `"110"` matter: unquoted, YAML would read it as a number.

```
admin@bastion — real capture

[admin@bastion ~]$ oc get storagemap vcenter-lab-storage -n openshift-mtv -o yaml | sed -n "/^spec:/,/^$/" | grep -vE "lastTransitionTime|observedGeneration"
spec:
  map:
    - destination:
        accessMode: ReadWriteMany
        storageClass: nfs-csi
        volumeMode: Filesystem
      source:
        name: DATASTORE-01
  provider:
    destination:
      name: host
      namespace: openshift-mtv
    source:
      name: vcenter-lab
      namespace: openshift-mtv
  status:
    conditions:
      - category: Required
        message: The storage map is ready.
        status: "True"
        type: Ready
    references:
      - id: datastore-1047
        name: DATASTORE-01
```

Every datastore that holds a disk of a VM in the plan must appear in the StorageMap, and every port group a NIC uses must appear in the NetworkMap, or the plan will not become Ready.

SCREENSHOT S-10:

Migration for Virtualization > Network maps and Storage maps: vcenter-lab-net (110 → default/vlan-110) and vcenter-lab-storage (DATASTORE-01 → nfs-csi), both Ready.

Step 4 · What a Plan can do

```
admin@bastion — real capture

[admin@bastion ~]$ oc explain plan.spec.type
GROUP:      forklift.konveyor.io
KIND:       Plan
VERSION:    v1beta1

FIELD: type <string>
ENUM:
  cold
  warm
  live
  conversion

DESCRIPTION:
  Migration type. e.g. "cold", "warm", "live", "conversion". Supersedes the
  `warm` boolean if set.
```

TYPE	WHAT HAPPENS	DOWNTIME	NEEDS
cold	Source powered off (by MTV if running), disks copied, converted, VM created	The whole copy	Nothing extra
warm	Disks copied while the source runs, deltas copied repeatedly via CBT, short final cutover	Minutes (final delta + conversion)	CBT on the source, VDDK in practice
live	Live migration between two OpenShift clusters (KubeVirt to KubeVirt)	None	Source is OpenShift Virtualization, not vSphere
conversion	Run only the guest conversion on disks already in place (for example after storage copy offload)	Depends	Disks already on OpenShift

```
[admin@bastion ~]$ oc explain plan.spec.targetPowerState
GROUP:      forklift.konveyor.io
KIND:       Plan
VERSION:    v1beta1

FIELD: targetPowerState <string>
ENUM:
  on
  off
  auto

DESCRIPTION:
  TargetPowerState specifies the desired power state of the target VM after
  migration.
  - "on": Target VM will be powered on after migration
  - "off": Target VM will be powered off after migration
  - "auto" or nil (default): Target VM will match the source VM's power state
```

TIP

`oc explain` works on every MTV and KubeVirt field and always matches the version installed on *your* cluster. When a blog post and `oc explain` disagree, believe `oc explain`.

PRODUCTION

One provider per vCenter, VDDK always, the vCenter CA instead of `insecureSkipVerify`, the MTV account scoped to the folders being migrated, one NetworkMap and StorageMap per source environment reused by every wave, and a dedicated **transfer network** (plan `transferNetwork`) so disk copies do not compete with VM traffic. For FC/iSCSI arrays with a supported vendor, use storage copy offload (`offloadPlugin.vsphereXcopyConfig` in the StorageMap).

EXAM TIP

For EX316 be able to create a provider, a NetworkMap, a StorageMap and a Plan both in the console and as YAML, start a Migration, and read why a Plan is not Ready from its conditions.

LAB — DO IT ON THE BASTION

Run `oc get provider vcenter-lab -n openshift-mtv -o jsonpath='{.status.conditions[?(@.type=="Ready")].message}'` and `oc get plan -n openshift-mtv -o custom-columns=NAME:.metadata.name,READY:.status.conditions[0].type`. Then open the provider in the console and count the VMs it sees: that number comes from the inventory container of `forklift-controller`, not from vCenter directly.

Check yourself

Q1. The provider shows `ConnectionInsecure=True`. Is the provider broken? No. It is a `Warn` category condition. *Logic:* only `Required` conditions decide readiness; this one records that TLS verification was skipped on purpose (lab vCenter has a self-signed certificate).

Q2. A plan stays not Ready and says a datastore is not mapped. The VM has two disks on two datastores. What do you do? Add the second datastore to the StorageMap. *Logic:* MTV must know a target StorageClass for every disk; one missing mapping blocks the whole plan.

Q3. Why does the NetworkMap use `type: multus` and not `type: pod`? The VM must keep an L2 presence on VLAN 110 like it had in VMware. *Logic:* `pod` gives a NATed IP on the cluster network (masquerade); `multus` attaches the VM to the bridge NAD with its own MAC on the real VLAN.

Q4. Which single setting would have made the lab migration much faster, and why? `spec.settings.vddkInitImage` on the provider. *Logic:* without VDDK, virt-v2v reads the disk through vCenter over HTTPS (`nbdkit curl`), around 2 to 3 MB/s here; with VDDK it reads directly from ESXi over NBD.

Q5. Can MTV damage the source VM in a cold migration? It powers it off if it is running, but it does not modify or delete it; rollback is "power the source back on". *Logic:* cold migration only reads the disks; nothing is written back to vSphere.

End to End: Migrating ubuntu-noble-24.04-cloudimg

This chapter is the whole migration of one real VM, in the order it happened on 5 October 2026. Do it once in the web console and once with `oc`; they create the same objects.

	SOURCE (VMWARE)	TARGET (OPENSIFT)
VM	ubuntu-noble-24.04-cloudimg , MoRef vm-2275 , powered off	ubuntu-noble-24.04-cloudimg in project mtv-test
CPU / RAM	2 vCPU, 1 GiB	2 sockets × 1 core, 1 GiB
Disk	10 GiB on datastore DATASTORE-01	PVC on nfs-csi , RWX, Filesystem
Network	port group VM-Network-110	NAD default/vlan-110 (bridge, VLAN 110)
Type		Cold, no VDDK

9.1 Create the target project

```
# Reference
[admin@bastion ~]$ oc new-project mtv-test
```

9.2 The plan, in the web console

1. Log in as `ocpadmin` . Open **Migration for Virtualization > Providers** and check `vcenter-lab` and `host` are both **Ready**.
2. **Migration plans > Create plan. Source provider:** `vcenter-lab` .
3. **Select VMs:** search `ubuntu` , tick `ubuntu-noble-24.04-cloudimg` . Look at the *Concerns* column: MTV's validation service flags problems here before you start.
4. **Plan name:** `test-ubuntu` . **Target provider:** `host` . **Target project:** `mtv-test` .
5. **Network map:** source `VM-Network-110` → target `default/vlan-110` . Do not choose *Pod network*.
6. **Storage map:** source `DATASTORE-01` → target `nfs-csi` .
7. **Migration type:** Cold. Create the plan and wait for **Ready**.

SCREENSHOT S-11:

Create plan, VM selection step: `ubuntu-noble-24.04-cloudimg` ticked, with its Concerns column.

SCREENSHOT S-12:

Create plan, maps step: `110` → `default/vlan-110` and `DATASTORE-01` → `nfs-csi`.

SCREENSHOT S-13:

Plan `test-ubuntu` details page, status *Ready*, with the *Start* button. A notice that VDDK is not configured is expected in this lab.

9.3 The same plan with `oc`

```

# Reference: ~/ocp-install/mtv-test/test-ubuntu-plan.yaml (NetworkMap + StorageMap + Plan)
apiVersion: forklift.konveyor.io/v1beta1
kind: NetworkMap
metadata: {name: vcenter-lab-net, namespace: openshift-mtv}
spec:
  provider:
    source: {name: vcenter-lab, namespace: openshift-mtv}
    destination: {name: host, namespace: openshift-mtv}
  map:
    - source: {name: "VM-Network-110"} # vSphere port group
      destination: {type: multus, namespace: default, name: vlan-110} # bridge br-vm, VLAN 110
  ---
apiVersion: forklift.konveyor.io/v1beta1
kind: StorageMap
metadata: {name: vcenter-lab-storage, namespace: openshift-mtv}
spec:
  provider:
    source: {name: vcenter-lab, namespace: openshift-mtv}
    destination: {name: host, namespace: openshift-mtv}
  map:
    - source: {name: DATASTORE-01} # vSphere datastore
      destination: {storageClass: nfs-csi, accessMode: ReadWriteMany, volumeMode: Filesystem}
  ---
apiVersion: forklift.konveyor.io/v1beta1
kind: Plan
metadata: {name: test-ubuntu, namespace: openshift-mtv}
spec:
  warm: false # cold
  targetNamespace: mtv-test
  provider:
    source: {name: vcenter-lab, namespace: openshift-mtv}
    destination: {name: host, namespace: openshift-mtv}
  map:
    network: {name: vcenter-lab-net, namespace: openshift-mtv}
    storage: {name: vcenter-lab-storage, namespace: openshift-mtv}
  vms:
    - name: ubuntu-noble-24.04-cloudimg

```

```

# Reference
[admin@bastion ~]$ oc apply -f ~/ocp-install/mtv-test/test-ubuntu-plan.yaml

```

Twelve lines of Plan were applied, but the API filled in the rest with defaults. This is what the cluster actually stored:

```
admin@bastion — real capture

[admin@bastion ~]$ oc get plan test-ubuntu -n openshift-mtv -o yaml | sed -n "/^spec:/,/^status:/p"
spec:
  deleteVmOnFailMigration: true
  map:
    network:
      name: vcenter-lab-net
      namespace: openshift-mtv
    storage:
      name: vcenter-lab-storage
      namespace: openshift-mtv
  migrateSharedDisks: true
  preserveStaticIPs: true
  provider:
    destination:
      name: host
      namespace: openshift-mtv
    source:
      name: vcenter-lab
      namespace: openshift-mtv
  pvcNameTemplateUseGenerateName: true
  runPreflightInspection: true
  skipGuestConversion: false
  targetNamespace: mtv-test
  useCompatibilityMode: true
  vms:
    - name: ubuntu-noble-24.04-cloudimg
  warm: false
  xfsCompatibility: false
status:
```

DEFAULT	MEANING	WHEN YOU CHANGE IT
<code>deleteVmOnFailMigration:</code> <code>true</code>	A failed run cleans up its half-built target VM	Set <code>false</code> to keep it for debugging
<code>preserveStaticIPs: true</code>	virt-v2v writes the source's static IP config onto the new NIC	Set <code>false</code> if the VM gets a new IP on OpenShift
<code>runPreflightInspection:</code> <code>true</code>	Inspect the guest before copying, fail fast on unsupported OS	Rarely
<code>skipGuestConversion:</code> <code>false</code>	Run virt-v2v (drivers, guest agent, bootloader)	<code>true</code> only for already-KVM-ready guests
<code>useCompatibilityMode:</code> <code>true</code>	Use SATA/E1000-style devices if virtio drivers might be missing	<code>false</code> once you trust the guest has virtio
<code>migrateSharedDisks: true</code>	Copy multi-writer disks with this plan	<code>false</code> when another plan owns the shared disk

9.4 Start it: the Migration object

```
# Reference: ~/ocp-install/mtv-test/test-ubuntu-migration.yaml
apiVersion: forklift.konveyor.io/v1beta1
kind: Migration
metadata: {name: test-ubuntu-1, namespace: openshift-mtv}
spec:
  plan: {name: test-ubuntu, namespace: openshift-mtv}
```

```
# Reference: equivalent to pressing Start
[admin@bastion ~]$ oc apply -f ~/ocp-install/mtv-test/test-ubuntu-migration.yaml
[admin@bastion ~]$ oc get migration test-ubuntu-1 -n openshift-mtv -w
```

NOTE

A Migration is one *run* of a plan. To run the same plan again (for example after fixing something), create a new Migration with a new name: `test-ubuntu-2`.

SCREENSHOT S-14:

Plan `test-ubuntu` while running: the VM row expanded, showing the pipeline steps with *ImageConversion* in progress.

9.5 Watching the pipeline

admin@bastion – real capture

```
[admin@bastion ~]$ oc get plan test-ubuntu -n openshift-mtv -o jsonpath="{range .status.migration.vms
[*].pipeline[*]}{.name}{\"\t\"}{.phase}{\"\t\"}{.started}{\"\t\"}{.completed}{\"\t\"}{.progress.complete
d}/{.progress.total}{\"\n\"}{end}" | column -t
```

Initialize	Completed	2026-10-05T15:08:05Z	2026-10-05T15:08:15Z	0/1
DiskAllocation	Completed	2026-10-05T15:08:15Z	2026-10-05T15:08:21Z	10240/10240
ImageConversion	Completed	2026-10-05T15:08:21Z	2026-10-05T15:31:47Z	0/1
DiskTransferV2v	Completed	2026-10-05T15:31:47Z	2026-10-05T16:32:29Z	10240/10240
VirtualMachineCreation	Completed	2026-10-05T16:32:29Z	2026-10-05T16:32:30Z	0/1

STEP	DURATION	WHAT HAPPENED
Initialize	10 s	Plan validated, VM looked up in the inventory
DiskAllocation	6 s	10 240 MB PVC created on <code>nfs-csi</code>
ImageConversion	23 min 26 s	virt-v2v pod inspected and converted the guest, reading through vCenter
DiskTransferV2v	60 min 42 s	The disk data copied, about 2.8 MB/s (no VDDK)
VirtualMachineCreation	1 s	<code>VirtualMachine</code> object created from the converted definition
Total	1 h 24 min 25 s	15:08:05 to 16:32:30 UTC

TIP

`progress 0/1` on Initialize, ImageConversion and VirtualMachineCreation is not a bug: those steps are counted as a single unit and only show complete/incomplete. Disk steps count megabytes.

```
admin@bastion - real capture

[admin@bastion ~]$ oc get plan test-ubuntu -n openshift-mtv -o jsonpath="{range .status.migration.vms [*]}{{.name}}{\n id=\">{.id}}{\n phase=\">{.phase}}{\n started=\">{.started}}{\n completed=\">{.completed}}{\n\n\">{end}"
ubuntu-noble-24.04-cloudimg id=vm-2275 phase=Completed started=2026-10-05T15:08:05Z completed=2026-10-05T16:32:30Z
```

The conversion Pod

MTV ran one Pod in the target project that did both conversion and copy:

```
admin@bastion - real capture

[admin@bastion ~]$ oc get pods -n mtv-test
NAME                READY   STATUS    RESTARTS   AGE
test-ubuntu-vm-2275-m2czj  0/1     Completed 0           5h53m
```

```
admin@bastion — real capture

[admin@bastion ~]$ oc get pod test-ubuntu-vm-2275-m2czj -n mtv-test -o jsonpath="{.spec.containers[*].image}{\"\n\"}{.status.startTime}{\"\n\"}"; oc logs test-ubuntu-vm-2275-m2czj -n mtv-test --tail=25
registry.redhat.io/migration-toolkit-virtualization/mtv-virt-v2v-rhel10@sha256:36266543303e3b78841c7894f7fc745ab8351507b638bc9766d0b378733180b5
2026-10-05T15:08:25Z
fsync /dev/sda
guestfsd: => internal_autosync (0x11a) took 0.04 secs
libguestfs: calling virDomainDestroy flags=VIR_DOMAIN_DESTROY_GRACEFUL
libguestfs: closing guestfs handle 0x5580566ff990 (state 0)
libguestfs: command: run: rm
libguestfs: command: run: \ -rf /tmp/libguestfsc3zBcz
libguestfs: command: run: rm
libguestfs: command: run: \ -rf /tmp/libguestfs2vsD5h
Starting server on :8080
2026/10/05 16:32:29 http: superfluous response.WriteHeader call from github.com/kubev2v/forklift/pkg/virt-v2v/server.Server.vmHandler (server.go:81)
2026/10/05 16:32:29 http: superfluous response.WriteHeader call from github.com/kubev2v/forklift/pkg/virt-v2v/server.Server.inspectorHandler (server.go:99)
Shutdown request received. Shutting down server.
```

Read it bottom-up. The image is `mtv-virt-v2v-rhel10` : `virt-v2v` runs in a RHEL 10 container. `libguestfs` is the library `virt-v2v` uses to open the guest disk inside a tiny appliance VM; `fsync /dev/sda` and `virDomainDestroy` are it flushing and closing that appliance. Then the Pod serves the converted VM definition on `:8080` , the controller fetches it at 16:32:29 (the exact second *VirtualMachineCreation* started), and the Pod shuts down. The two `superfluous response.WriteHeader` lines are harmless Go HTTP warnings.

GOTCHA — REAL ISSUE

No VDDK = slow. During the copy the same Pod's log showed `nbdkit: curl[4]: ...` and `virt-v2v monitoring: Progress update, completed 17 %` : the disk was being read over HTTPS through vCenter with `nbdkit`'s `curl` plug-in. 10 GiB took 61 minutes; a 500 GiB production disk would take more than two days. Configure VDDK (Chapter 8) before any real wave.

9.6 What MTV built

```
admin@bastion — real capture

[admin@bastion ~]$ oc get plan,migration -n openshift-mtv
NAME                                READY  EXECUTING  SUCCEEDED  FAILED  AGE
plan.forklift.konveyor.io/test-ubuntu  True
NAME                                READY  RUNNING  SUCCEEDED  FAILED  AGE
migration.forklift.konveyor.io/test-ubuntu-1  True
```

```
admin@bastion — real capture
```

```
[admin@bastion ~]$ oc get plan test-ubuntu -n openshift-mtv -o jsonpath="{range .status.conditions[*]}{.type}{\"\t\"}{.status}{\"\t\"}{.message}{\"\n\"}{end}"
Ready    True    The migration plan is ready.
Succeeded True    The plan execution has SUCCEEDED.
```

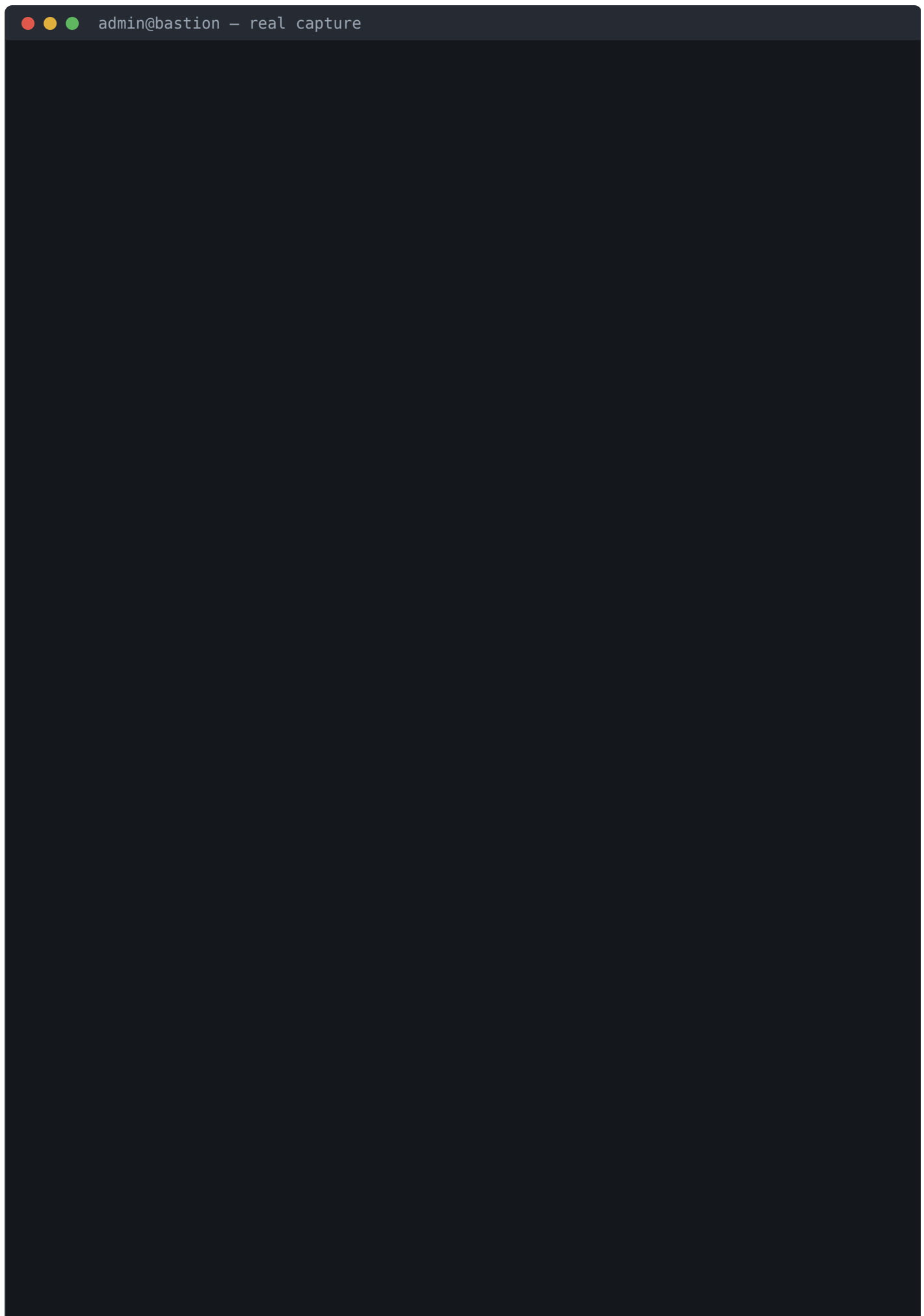
```
admin@bastion — real capture
```

```
[admin@bastion ~]$ oc get vm,vmi -n mtv-test; oc get pvc,dv -n mtv-test
```

NAME	AGE	STATUS	READY
virtualmachine.kubevirt.io/ubuntu-noble-24.04-cloudimg	4h29m	Stopped	False

NAME	STATUS	VOLUME	CA	
PACITY	ACCESS MODES	STORAGECLASS	VOLUMEATTRIBUTESCLASS	AGE
persistentvolumeclaim/test-ubuntu-vm-2275-dm9s9	Bound	pvc-e18f93ca-a91e-46e9-9306-2bda130b4efd	11	
381663335	RWX	nfs-csi	<unset>	5h53m

The VM is `Stopped` because the source was powered off and `targetPowerState` defaults to `auto` (match the source). There is no VMI (not running) and no DataVolume: MTV fills PVCs directly with a volume populator. The PVC name `test-ubuntu-vm-2275-dm9s9` is *plan name - source MoRef - random suffix* (`pvcNameTemplateUseGenerateName: true`).



```
[admin@bastion ~]$ oc get vm ubuntu-noble-24.04-cloudimg -n mtv-test -o yaml | sed -n "/^spec:/,/^status:/p" | head -80
spec:
  preference:
    kind: virtualmachineclusterpreference
    name: linux
  runStrategy: Halted
  template:
    metadata:
      annotations:
        kubevirt.io/pci-topology-version: v3
        vm.kubevirt.io/flavor: ""
        vm.kubevirt.io/os: ""
        vm.kubevirt.io/workload: ""
      labels:
        app: ubuntu-noble-24.04-cloudimg
    spec:
      architecture: amd64
      domain:
        clock:
          timezone: UTC
        cpu:
          cores: 1
          sockets: 2
        devices:
          disks:
            - bootOrder: 1
              disk:
                bus: virtio
                name: vol-0
                serial: 6000C290-0000-0000-0
            - disk:
                bus: virtio
                name: cloudinitdisk
          inputs:
            - bus: virtio
              name: tablet
              type: tablet
          interfaces:
            - bridge: {}
              macAddress: 00:50:56:aa:00:01
              model: virtio
              name: net-0
          tpm:
            enabled: false
        firmware:
          bootloader:
            bios: {}
            serial: VMware-42 00 00 00 00 00 00 00-00 00 00 00 00 00 00 01
            uuid: f9b9a01b-3c5d-4463-a614-92cb4ba5b3f5
        machine:
          type: pc-q35-rhel9.8.0
        memory:
          guest: 1Gi
          resources: {}
        networks:
          - multus:
              networkName: default/vlan-110
              name: net-0
        volumes:
          - name: vol-0
            persistentVolumeClaim:
```

```

    claimName: test-ubuntu-vm-2275-dm9s9
  - cloudInitNoCloud:
      secretRef:
        name: ubuntu-noble-24.04-cloudimg-cloudinit
      name: cloudinitdisk
status:

```

Every field traced back to its source:

FIELD	VALUE	WHERE IT CAME FROM
runStrategy	Halted	Source was powered off, targetPowerState: auto
cpu	2 sockets × 1 core	Source vCPU topology, copied exactly
memory.guest	1Gi	Source RAM
firmware.bootloader	bios	Source firmware (BIOS, not EFI)
firmware.serial	VMware-42 00 00 ...	The VMware BIOS serial, kept so licence checks keyed to it still pass
disks[vol-0].serial	6000C290-...	The VMDK's UUID, kept so /dev/disk/by-id paths in the guest do not change
interfaces[net-0].macAddress	00:50:56:aa:00:01	The source NIC's VMware MAC, kept so DHCP reservations and MAC-bound configs keep working
interfaces[net-0].bridge , networks.multus	default/vlan-110	The NetworkMap
disk bus: virtio		virt-v2v installed virtio drivers, so it used the fast bus
volumes[vol-0]	PVC test-ubuntu-vm-2275-dm9s9	The StorageMap
preference: linux		Chosen by MTV from the detected guest OS
cloudinitdisk volume	Secret ...-cloudinit	Not from MTV: added afterwards to give the cloud image a login (next section)

GOTCHA — REAL ISSUE

The test-ubuntu README predicted that MTV would rename the VM (dots to dashes). The capture shows it did not: `ubuntu-noble-24.04-cloudimg` is a valid Kubernetes name, because dots are allowed in object names. MTV only renames VMs whose names are not valid (uppercase letters, spaces, underscores, longer than 63 characters). Check `oc get vm -n <project>` rather than guessing.

9.7 First boot

The Ubuntu *cloud image* has no password at all (it expects cloud-init), so the VM was given a login before its first start:

```
# Reference: ~/ocp-install/mtv-test/add-cloudinit-login.sh (prompts; only a SHA-512 hash is stored)
HASH=$(openssl passwd -6 "$PW")
oc create secret generic ubuntu-noble-24.04-cloudimg-cloudinit -n mtv-test --from-literal=userdata="#cloud-config
users:
- name: ubuntu
  lock_passwd: false
  passwd: '$HASH'
  sudo: ALL=(ALL) NOPASSWD:ALL
ssh_pwauth: true"
oc patch vm ubuntu-noble-24.04-cloudimg -n mtv-test --type=json -p '[
  {"op": "add", "path": "/spec/template/spec/domain/devices/disks/-", "value": {"name": "cloudinitdisk", "disk":
{"bus": "virtio"}}},
  {"op": "add", "path": "/spec/template/spec/volumes/-", "value": {"name": "cloudinitdisk", "cloudInitNoCloud":
{"secretRef": {"name": "ubuntu-noble-24.04-cloudimg-cloudinit"}}}]'
```

admin@bastion — real capture

```
[admin@bastion ~]$ oc get secret -n mtv-test | grep -vE "dockercfg|token"
NAME                                TYPE      DATA   AGE
ubuntu-noble-24.04-cloudimg-cloudinit  Opaque    1       41m
```

Then the VM was started from the console (*Virtualization > VirtualMachines > mtv-test > ubuntu-noble-24.04-cloudimg > Start*), checked in the *Console* tab, and shut down again. The project's events are the proof that it really booted on OpenShift:

```
admin@bastion — real capture

[admin@bastion ~]$ oc get events -n mtv-test --sort-by=.lastTimestamp | head -25
LAST SEEN   TYPE      REASON          OBJECT                                                                 MESSAGE
37m         Normal    Scheduled       pod/virt-launcher-ubuntu-noble-24.04-cloudimg-cstks    Successfully assigned mtv-test/virt-launcher-ubuntu-noble-24.04-cloudimg-cstks to ocp-node-1
37m         Normal    SuccessfulCreate virtualmachineinstance/ubuntu-noble-24.04-cloudimg    Created virtual machine pod virt-launcher-ubuntu-noble-24.04-cloudimg-cstks
37m         Normal    SuccessfulCreate virtualmachine/ubuntu-noble-24.04-cloudimg            Started the virtual machine by creating the new virtual machine instance ubuntu-noble-24.04-cloudimg
37m         Normal    Started        pod/virt-launcher-ubuntu-noble-24.04-cloudimg-cstks    Container started
37m         Normal    Created        pod/virt-launcher-ubuntu-noble-24.04-cloudimg-cstks    Container created
37m         Normal    Pulled         pod/virt-launcher-ubuntu-noble-24.04-cloudimg-cstks    Container image "registry.redhat.io/container-native-virtualization/virt-launcher-rhel9@sha256:a6e0242c8771071b8aa5d36193e9513740fde96ef560c6a1ff81a5e4ed78bbec" already present on machine and can be accessed by the pod
37m         Normal    AddedInterface pod/virt-launcher-ubuntu-noble-24.04-cloudimg-cstks    Add pod900d2c01c91 [] from default/vlan-110
37m         Normal    AddedInterface pod/virt-launcher-ubuntu-noble-24.04-cloudimg-cstks    Add eth0 [10.130.0.92/23] from ovn-kubernetes
...
37m         Normal    Started        virtualmachineinstance/ubuntu-noble-24.04-cloudimg    VirtualMachineInstance started.
37m         Normal    Created        virtualmachineinstance/ubuntu-noble-24.04-cloudimg    VirtualMachineInstance defined.
35m         Normal    ShuttingDown   virtualmachineinstance/ubuntu-noble-24.04-cloudimg    Signaled Graceful Shutdown
35m         Normal    SuccessfulDelete virtualmachine/ubuntu-noble-24.04-cloudimg            Stopped the virtual machine by deleting the virtual machine instance 477a1f54-2b09-43f8-9d6a-9490393a2387
...
34m         Normal    Stopped        virtualmachineinstance/ubuntu-noble-24.04-cloudimg    The VirtualMachineInstance was shut down.
34m         Normal    SuccessfulDelete virtualmachineinstance/ubuntu-noble-24.04-cloudimg    Deleted virtual machine pod virt-launcher-ubuntu-noble-24.04-cloudimg-cstks
```

Read it as a boot log of the *platform*:

1. *Started the virtual machine by creating the new virtual machine instance*: someone pressed Start; the VM controller created a VMI.
2. *Successfully assigned ... to ocp-node-1*: the scheduler placed the `virt-launcher` Pod on node 1 (it had `devices.kubevirt.io/kvm` and `br-vm`).
3. *Add eth0 [10.130.0.92/23] from ovn-kubernetes*: every Pod also gets a Pod-network interface; the VM does not use it here.
4. *Add pod900d2c01c91 [] from default/vlan-110*: **the VM's NIC was plugged into the VLAN 110 bridge**. The empty `[]` means OpenShift assigned no IP (`ipam: {}`): the guest gets its address itself, as it did on VMware.
5. *VirtualMachineInstance started*: QEMU/KVM is running the guest inside the Pod.
6. Two minutes later, *Signaled Graceful Shutdown* and *Stopped the virtual machine by deleting the virtual machine instance*: someone pressed Stop; the guest got an ACPI shutdown, then the Pod was removed.

```
admin@bastion — real capture

[admin@bastion ~]$ oc get vm ubuntu-noble-24.04-cloudimg -n mtv-test -o jsonpath="{.status.printableStatus}{\\n\\n}{range .status.conditions[*]}{.type}={.status} {.reason}{\\n\\n}{end}"
Stopped
Ready=False VMINotExists
LiveMigratable=True
StorageLiveMigratable=True
```

Ready=False VMINotExists is just "powered off". The two important lines are LiveMigratable=True and StorageLiveMigratable=True : thanks to the RWX StorageMap, this VM can be moved between nodes while running (vMotion) and between StorageClasses (Storage vMotion).

SCREENSHOT S-15:
Virtualization > VirtualMachines > mtv-test > ubuntu-noble-24.04-cloudimg, Overview tab while running: node ocp-node-1 , network vlan-110 , disk on nfs-csi .

SCREENSHOT S-16:
Console tab: Ubuntu 24.04 login prompt, then ip a showing the NIC with MAC 00:50:56:aa:00:01 .

9.8 Validate, roll back, clean up

CHECK	HOW	EXPECTED
Boots	Console tab	Login prompt
Same MAC	ip link in the guest	00:50:56:aa:00:01
Network	ip a , ping <gateway>	Address on the VLAN 110 subnet, gateway answers (if the VLAN has DHCP or a static IP was preserved)
Guest agent	VM Overview shows OS name and IP	virt-v2v installed qemu-guest-agent
Disk	lsblk in the guest	10 GiB vda
Application	Your smoke test	Same as on VMware

Rollback for a cold migration is: stop the OpenShift VM, power the source VM back on in vSphere. Nothing on the VMware side was changed.

```
# Reference: clean up when done (keep the maps for the next VM)
oc delete project mtv-test
oc delete plan/test-ubuntu -n openshift-mtv
oc project default
```

EXAM TIP

EX316 can ask you to migrate a VM and then prove it: read the plan's pipeline, find the VM, start it, attach it to the right network, and show it is live-migratable. Everything in this chapter is the answer key.

LAB — DO IT ON THE BASTION

Migrate the same VM again as `test-ubuntu-2` into a project `mtv-test2`, but this time set `targetPowerState: "on"` and `preserveStaticIPs: false` in the plan. Compare the pipeline timings and the new VM's `runStrategy` with the first run.

Check yourself

Q1. The pipeline shows `DiskTransferV2v Running 1740/10240` for a long time. Is it stuck?

Not necessarily. *Logic:* without VDDK the copy runs at a few MB/s; check that the number keeps rising (`-w`) and read the conversion Pod's log for `virt-v2v monitoring: Progress update`.

Q2. Why does the migrated VM keep the VMware MAC `00:50:56:...`, and when would that be a problem? MTV copies the source MAC so DHCP reservations and MAC-bound guest configs keep working. *Logic:* it becomes a problem only if the source VM is powered on again on the same VLAN at the same time (duplicate MAC). Never run both.

Q3. The VM is `Stopped` right after a successful migration. What decided that?

`targetPowerState: auto` with a source that was powered off. *Logic:* `auto` = match the source; set `on` in the plan to start it automatically.

Q4. In the events, the virt-launcher Pod got `eth0 10.130.0.92/23` from OVN-Kubernetes. Does the VM use that address? No. *Logic:* the VM's only interface `net-0` is a bridge on `default/vlan-110`; the Pod's `eth0` belongs to the launcher container itself.

Q5. How do you run the same plan a second time? Create a new Migration object with a new name pointing at the plan. *Logic:* a Migration is a single, immutable run; its status is history.

Troubleshooting and Going to Production

The troubleshooting order

When something does not work, go from the platform down to the object, and read before you change.

```
oc get co | awk 'NR==1 || $3!="True" || $5!="False"' # 1. is the platform healthy?
oc get nodes; oc adm top nodes # 2. are the nodes?
oc get csv -A | grep -v Succeeded # 3. are the operators?
oc get hco -n openshift-cnv -o jsonpath='{.status.conditions[?(@.type=="Degraded")].status}'
oc get providers,networkmap,storagemap,plan -n openshift-mtv # 4. MTV objects Ready?
oc describe plan <plan> -n openshift-mtv | sed -n '/Conditions/, $p' # 5. why not?
oc get pods,pvc,events -n <target-project> --sort-by=.lastTimestamp # 6. what happened in the target
oc logs <conversion-pod> -n <target-project> --tail=50 # 7. the conversion itself
```

Symptom table

SYMPTOM	LIKELY CAUSE	CHECK	FIX
Provider not Ready, <i>ConnectionTestSucceeded=False</i>	Wrong password, DNS, or 443 blocked	<code>oc describe provider ;</code> from a node: <code>curl -k https://<vcenter>/sdk</code>	Fix the connection or firewall
Provider stuck loading inventory	Very large vCenter or account cannot see objects	<code>oc logs deploy/forklift-controller -c inventory -n openshift-mtv</code>	Grant right
Plan not Ready: <i>network not mapped / storage not mapped</i>	A NIC's port group or a disk's datastore missing from the map	Plan conditions	Add mapping
Plan warning: <i>VM has snapshots</i>	Existing snapshots on the source	Validation concerns column	Delete snapshots
<code>DiskTransfer</code> very slow	No VDDK; or 902 blocked to the ESXi host	Provider <code>settings</code> ; port test from Chapter 2	Add VDDK
<code>ImageConversion</code> failed	Unsupported guest OS, encrypted disk, BitLocker	Conversion Pod log	Fix the image (suspend, upgrade, skip for KVM)
VM created but will not schedule	No node with KVM, or NAD bridge missing on nodes	<code>oc describe vmi ; oc get nodes -o custom-columns=...kvm</code>	Expose virtio NNCI
VM boots, no network	VLAN ID wrong, trunk/promiscuous settings (nested), DHCP missing	NNCE status, NAD <code>vlan</code> , vSphere port group	Correct group static
VM not live-migratable	RWO disk	<code>oc get vm -o jsonpath='{.status.conditions}'</code> → <code>LiveMigratable=False</code>	Storage Storage
<code>oc debug node</code> fails with <i>namespace ... not found</i>	Context points to a deleted project	<code>oc project</code>	<code>oc p</code> or <code>-</code> names

SYMPTOM	LIKELY CAUSE	CHECK	FIX
Node stunned, etcd alarms after vSphere changes	ISO or NIC changed on a running node	Node journal gap	Change hardware power

Support data

```
# Reference: platform must-gather, and the MTV/Virtualization ones (the images come from the installed C
SVs)
oc adm must-gather --dest-dir=~ /mg-platform
oc adm must-gather --dest-dir=~ /mg-cnv \
  --image=$(oc get csv -n openshift-cnv -o jsonpath='{.items[0].spec.relatedImages[?(@.name=="must-gathe
r")].image}')
oc adm must-gather --dest-dir=~ /mg-mtv \
  --image=$(oc get csv -n openshift-mtv -o jsonpath='{.items[0].spec.relatedImages[?(@.name=="must_gathe
r")].image}')
```

```
admin@bastion - real capture
```

```
[admin@bastion ~]$ oc get csv -n openshift-mtv -o jsonpath="{.items[0].spec.relatedImages[*].name}" | tr
" " "\n" | grep -i must
must_gather
```

The MTV operator ships its own must-gather image, named `must_gather` (underscore) in its CSV. Querying the CSV means you always get the image matching the installed version, which is what Red Hat support will ask for.

From lab to production

AREA	THIS LAB	PRODUCTION
Nodes	3 nested VMs, compact	3 control plane + bare-metal workers sized like ESXi hosts
Storage	One NFS VM, <code>nfs-csi</code>	Array vendor CSI or ODF, block, RWX, snapshots; storage copy offload where supported
Networks	linux bridge on one NIC	Bonded NICs, one NAD per VLAN, dedicated migration and transfer networks
Identity	htpasswd <code>ocpadmin</code>	AD/LDAP or OIDC, group-based RBAC
vCenter account	Propagated on the root	Scoped to migrated folders, password in a vault
TLS	<code>insecureSkipVerify</code>	vCenter CA trusted
Disk transfer	No VDDK	VDDK always; warm migration for large VMs
Process	One test VM	Waves: pilot (non-critical) → wave per application, each with a change record, baseline, cutover window, validation, rollback
Backup	None	OADP (Velero) for VMs and their PVCs, tested restore before wave 1

A wave, step by step

1. **Discover:** list VMs per application (RVTools or the MTV inventory), record OS, disks, NICs, IPs, dependencies.
2. **Prepare:** remove snapshots, enable CBT (warm), suspend BitLocker, record the baseline (IP, MAC, services).
3. **Plan:** one Plan per application or wave, warm for anything large, `targetPowerState: on`.
4. **Pre-copy** (warm): start the migration days ahead; MTV copies deltas on a schedule (`controller_precopy_interval`, 60 minutes by default).
5. **Cutover:** in the change window, set the cutover time; MTV powers the source off, copies the last delta, converts, starts the VM.
6. **Validate:** the table in 9.8, plus the application owner's test.
7. **Decide:** keep, or roll back by powering the source on. Delete source VMs only after the agreed soak period.

EXAM TIP

Red Hat exams reward reading status, not guessing. For every task, finish with a `get` and a `describe` that prove the result, exactly like the real captures in this book.

LAB — DO IT ON THE BASTION

Break and fix, one at a time, on a scratch plan (never on the provider other people use): (1) remove the datastore from a copy of the StorageMap and read the plan error; (2) point a NAD at VLAN 9999 and boot a VM on it; (3) set your context to a project you then delete, and reproduce the `oc debug` error from Chapter 4.

Check yourself

Q1. In what order do you look when "the migration failed"? Platform (ClusterOperators, nodes), operators (CSV, HCO), MTV objects (provider, maps, plan conditions), then the target project (Pods, PVCs, events, conversion log). *Logic:* each layer depends on the one above; a failure higher up explains everything below it.

Q2. Warm migration: what makes the cutover short? Most of the data is copied while the source runs; at cutover only the last delta (changed blocks via CBT) is copied before conversion. *Logic:* downtime = final delta + conversion + boot, not the full disk copy.

Q3. Why collect must-gather with the image from the installed CSV instead of `latest`? It must match the operator version that produced the problem. *Logic:* support compares your data with that version's code and known issues.

Command Cheat Sheets

Cluster

TASK	COMMAND
Who and where am I	<code>oc whoami; oc whoami --show-server; oc project</code>
Version and health	<code>oc get clusterversion; oc get co</code>
Unhealthy operators only	<code>oc get co awk 'NR==1 \$3!="True" \$5!="False"'</code>
Nodes, usage	<code>oc get nodes -o wide; oc adm top nodes</code>
Node shell	<code>oc debug node/<node> --to-namespace=default -- chroot /host</code>
Installed operators	<code>oc get csv -A; oc get subscription -A</code>
Field reference	<code>oc explain <kind>.<field></code>
Events, newest last	<code>oc get events -n <ns> --sort-by=.lastTimestamp</code>

Storage

TASK	COMMAND
Classes and default	<code>oc get sc</code>
Claims and volumes	<code>oc get pvc -A; oc get pv</code>
CDI's view of a class	<code>oc get storageprofile <sc> -o yaml</code>
DataVolumes and progress	<code>oc get dv -A</code>
Grow a disk	<code>oc patch pvc <pvc> -n <ns> -p '{"spec":{"resources":{"requests":{"storage":"20Gi"}}}}'</code>

Virtualization

TASK	COMMAND
VMs and running instances	<code>oc get vm,vmi -A</code>
Start / stop	<code>virtctl start <vm> / virtctl stop <vm> (or patch runStrategy)</code>
Console	<code>virtctl console <vm> , virtctl vnc <vm></code>
Live migrate	<code>virtctl migrate <vm> ; oc get vmim -A</code>
Where is it	<code>oc get vmi <vm> -o wide</code>
KVM capacity per node	<code>oc get nodes -o custom-columns=NODE:.metadata.name,KVM:.status.allocatable.devices\.kubevirt\.io/kvm</code>
Boot sources	<code>oc get datasource -n openshift-virtualization-os-images</code>
Sizes and OS profiles	<code>oc get virtualmachineclusterinstancetype; oc get virtualmachineclusterpreference</code>
Platform health	<code>oc get hco -n openshift-cnv -o jsonpath='{range .status.conditions[*]}{.type}={.status}{"\n"}{end}'</code>

VM networking

TASK	COMMAND
Policies and per-node result	<code>oc get nncp; oc get nnce</code>
A node's live interfaces	<code>oc get nns <node> -o yaml</code>
VM networks	<code>oc get net-attach-def -A</code>
A NAD's config	<code>oc get net-attach-def <nad> -n <ns> -o jsonpath='{.spec.config}' python3 -m json.tool</code>

MTV

TASK	COMMAND
Everything	<code>oc get providers,networkmap,storagemap,plan,migration -n openshift-mtv</code>
Why not Ready	<code>oc get <kind> <name> -n openshift-mtv -o jsonpath='{range .status.conditions[*]}{.type}={.status} {.message}{"\n"}{end}'</code>
Pipeline	<code>oc get plan <plan> -n openshift-mtv -o jsonpath='{range .status.migration.vms[*].pipeline[*]}{.name} {.phase} {.progress.completed}/{.progress.total}{"\n"}{end}'</code>
Start	<code>oc create -f migration.yaml</code> (new name each run)
Add VDDK	<code>oc patch provider <p> -n openshift-mtv --type merge -p '{"spec":{"settings":{"vddkInitImage":"<image>"}}}'</code>
Inventory log	<code>oc logs deploy/forklift-controller -c inventory -n openshift-mtv --tail=50</code>

Bastion (Linux)

TASK	COMMAND
OS and kernel	<code>cat /etc/os-release; uname -r</code>
IPs, routes, DNS	<code>ip -br addr; ip route; cat /etc/resolv.conf</code>
Which route for an IP	<code>ip route get <ip></code>
DNS lookup	<code>dig +short <name></code>
Port test	<code>timeout 4 bash -c "</dev/tcp/<host>/<port>" && echo open</code>
Disks	<code>lsblk -o NAME,SIZE,TYPE,FSTYPE,LABEL,MOUNTPOINTS; df -h</code>
NFS exports	<code>sudo exportfs -v; showmount -e localhost</code>
Firewall	<code>sudo firewall-cmd --list-services</code>
Time	<code>chronyc sources</code>

Exam Map

EXAM	WHAT IT IS	WHERE IN THIS BOOK
DO180 (Red Hat OpenShift Administration I)	Course: oc , projects, Pods, Deployments, Routes, storage basics	Ch 3 to 5 (as foundation)
EX188 / DO188 (Containers and Podman)	Exam: build, run and manage containers with Podman	Ch 8 (VDDK image build with podman)
DO280 / EX280 (OpenShift Administrator)	Exam: identity providers, RBAC, operators, networking, storage, cluster health	Ch 3, 4, 5, 6 (operators), 10
DO316 / EX316 (OpenShift Virtualization Specialist)	Exam: install Virtualization, VMs, disks, networks, live migration, import/migrate VMs	Ch 6, 7, 8, 9
EX380 / EX370 / EX430 / EX432 / EX267	Automation, ODF, ACS, ACM, OpenShift AI	Not in this edition

NOTE

This edition covers the migration path only. The same format can carry the full exam track (one chapter per DO180, DO280, EX188, EX288, EX316, EX370, EX380, EX430, EX432, EX267 objective set) captured on the same cluster.